

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Gestión de Soluciones - I

0.1. Introducción: de los datos al conocimiento

El **dato** es una representación sintáctica, generalmente numérica, que puede manejar un dispositivo electrónico - normalmente un ordenador - sin significado por sí solo. Sin embargo, el dato es a su vez el ingrediente fundamental y el elemento de entrada necesario en cualquier sistema y/o proceso que pretenda extraer información o conocimiento sobre un dominio determinado. En este sentido, 7 es un dato, como también lo es π o como son los términos *aprobado* o *suspenso*.

Por su parte, la **información** es el dato interpretado, es decir, el dato con significado. Para obtener información, ha sido necesario un proceso en el que, a partir de un dato como elemento de entrada, se realice una **interpretación** de ese dato que permita obtener su significado, es decir, información a partir de él. La información es también el elemento de entrada y de salida en cualquier proceso de toma de decisiones. Partiendo de los datos del ejemplo anterior, información obtenida a partir de los mismos puede ser: *El 7 es un número primo, π es una constante cuyo valor es 3,141592653..., María ha aprobado el examen de conducir, Pablo está suspenso en matemáticas.*

A partir de información, es posible construir **conocimiento**. El conocimiento es información aprendida, que se traduce a su vez en reglas, asociaciones, algoritmos, etc. que permiten resolver el proceso de toma de decisiones. Así pues, la información obtenida a partir de los datos permite generar conocimiento, es decir, aprender. El conocimiento no es estático, como tampoco lo es siempre el aprendizaje. Aprender, construir conocimiento, implica necesariamente contrastar y validar el conocimiento construido con nueva información que permita, a su vez, guiar el aprendizaje y construir conocimiento nuevo. Siguiendo con los ejemplos anteriores, el conocimiento que permite obtener que el 7 es un número primo puede ser

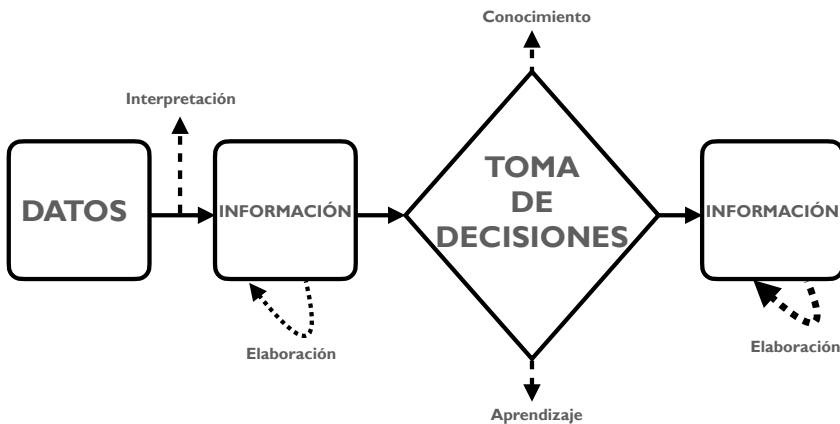


Figura 1: Relación entre datos, información y conocimiento en el proceso de toma de decisiones.

el *algoritmo de Eratóstenes*. Por otra parte, el conocimiento que permite obtener el valor del número π puede extraerse de los resultados de los trabajos de *Jones*, *Euler* o *Arquímedes*, mientras que el aprobado de María en el examen de conducir y el suspenso de Pablo en matemáticas, se pueden obtener de la regla que en una escala de diez asigna el aprobado a notas mayores o iguales que 5 y el suspenso a notas menores.

Por tanto, datos, información y conocimiento están estrechamente relacionados entre sí y dirigen cualquier proceso de **toma de decisiones** (ver [AN95], [FPSS96]). La figura 1 muestra la relación entre datos, información y conocimiento, en un proceso genérico de toma de decisiones. Más concretamente, en el ejemplo del suspenso de Pablo en matemáticas, el proceso de toma de decisión acerca de la calificación de Pablo se estructuraría de la siguiente forma:

1. El profesor corrige el examen de Pablo, que ha sacado un 3. Esta calificación, por sí sola, es simplemente un dato.
2. A continuación, el profesor calcula la calificación final de Pablo, en base a la nota del examen, sus trabajos y prácticas de laboratorio. *La nota final de Pablo es un 4*. Esto último es información.
3. ¿Ha aprobado Pablo? La información de entrada al proceso de decisión es su calificación final de 4 puntos, obtenida en el paso anterior. El conocimiento del profesor sobre el sistema de calificación le indica que una nota menor a 5 puntos se corresponde con un suspenso y, en caso contrario, con un aprobado.
4. La información de salida tras este proceso de decisión es que *Pablo está suspenso en matemáticas*.



Pregunta. Siguiendo el ejemplo anterior ¿Cómo se produce el proceso de toma de decisiones para determinar si un número es primo?

Aunque se trate de un ejemplo trivial, la importancia del proceso de toma de decisiones no lo es. En **marketing**, por ejemplo, se analizan bases de datos de clientes para identificar distintos grupos e intentar predecir el comportamiento de estos. En el mundo de las **finanzas**, las inversiones realizadas por grandes empresas responden a un proceso complejo de **toma de decisiones** donde los datos son el eje fundamental de este proceso. En **medicina**, existe una gran cantidad de sistemas de ayuda a la decisión que permiten a los doctores contrastar y validar sus diagnósticos de forma precoz. En definitiva, no hay área de conocimiento ni ámbito de aplicación que escape al proceso de toma de decisiones.

0.2. La carrera entre los datos y la tecnología

Que los datos son el elemento fundamental en cualquier proceso y/o sistema de **toma de decisiones** no es algo nuevo. Sin embargo, los datos no siempre han estado al alcance de los expertos y no siempre ha sido posible ni sencillo procesarlos según las necesidades concretas de cada caso de aplicación.

La información, por tanto, siempre ha sido poder y el gran reto ha sido y sigue siendo extraer información a partir de datos para generar conocimiento. Para ello, es necesario contar con dos factores que deben estar alineados: **datos y tecnología**.

Obtener **datos** no ha sido siempre una tarea fácil. Esto es debido principalmente a que la gran cantidad de sensores disponibles en la actualidad, que permiten registrar magnitudes de cualquier proceso, no existía como a día de hoy. Además, los sensores existentes en esta época (finales del siglo XX y comienzos del siglo XXI) no estaban ampliamente extendidos, ya que sus prestaciones estaban lejos de las que ofrecen hoy y sus precios no estaban al alcance de cualquier usuario. Por tanto, los procesos que se monitorizaban y de los cuales se recogían datos eran, sobretudo, procesos industriales realizados en grandes empresas. Por todos estos motivos, tradicionalmente se recurría a **modelos de simulación** que, a través de la implementación de un modelo matemático, permitían generar datos realistas de un proceso. Así, por ejemplo, se han generado datos de consumo eléctrico, tráfico urbano... Los datos generados mediante simulación son conocidos como **datos sintéticos** mientras que los datos provenientes de las lecturas de un sensor se conocen como **datos reales**.

Pero con los datos no es suficiente. Es necesario también contar con la **tecnología** necesaria para su procesamiento. Generar, almacenar y procesar todos estos datos no es una tarea trivial. Así pues, el **almacenamiento** se presenta como el primer problema tecnológico a resolver. Algunas soluciones propuestas pasan por los

sistemas de información distribuida, entendidos como un conjunto de ordenadores separados físicamente y conectados en red destinados al almacenamiento de datos o por los **sistemas de información en la nube**, que permiten adquirir espacio de almacenamiento en servidores privados, dejando la gestión de estos servidores en manos del proveedor. El segundo problema tecnológico es el **procesamiento** de los datos almacenados. Este aspecto cobra especial relevancia en función del caso de aplicación, pudiendo distinguirse entre procesamiento *on-line* (en línea) y procesamiento *off-line* (fuera de línea). En el caso del procesamiento *on-line*, los datos son procesados a medida que son generados, ya que se requiere una respuesta en tiempo real. Por ejemplo, en un sistema de control del tráfico que permite regular los semáforos en función del tráfico actual, el sistema debe regular el semáforo a medida que se van generando e interpretando los datos del tráfico en un instante de tiempo dado. Por otra parte, en el caso del procesamiento *off-line*, no es necesario que los datos se procesen a medida que se generan. Por ejemplo, en un sistema de detección del fraude bancario, comprobar si un cliente ha realizado algún movimiento fraudulento es una tarea que puede llevarse a cabo *off-line*, por ejemplo, haciendo un análisis de los movimientos del cliente en un momento dado, sin tener por qué diagnosticar cada movimiento que este va realizando. En este sentido, la **computación distribuida**, en donde múltiples máquinas realizan el procesamiento optimizando el rendimiento o la **computación en la nube**, que permite adquirir recursos de procesamiento al igual que se puede adquirir espacio de almacenamiento, son dos soluciones al problema del procesamiento. Otras alternativas son la **programación paralela** y la **programación multi-procesador**, que permiten, respectivamente, aprovechar el paralelismo de múltiples hilos de ejecución dentro de un procesador y realizar el procesamiento dividiéndolo en múltiples hilos en diferentes procesadores (ver [MSC13]).



Pregunta. Piensa en procesos cotidianos que requieran un procesamiento *on-line* y en otros que requieran un procesamiento *off-line*.

En la actualidad, la proliferación de una gran cantidad de sensores con altas prestaciones y precios asequibles que permiten monitorizar y generar datos sobre cualquier proceso ha supuesto un **incremento exponencial en la cantidad de datos generados**. En la actualidad, es posible monitorizar casi cualquier proceso, incluyendo los domésticos como el consumo eléctrico de un hogar, la presencia dentro del mismo o procesos cotidianos como la actividad física, entre otros muchos. Hoy, los datos llevan la delantera en la carrera entre datos y tecnología. Si bien es cierto que la tecnología ha experimentado grandes avances en los últimos años, la cantidad de datos generada no deja de crecer. Esto supone un **reto permanente para la tecnología**, que sigue evolucionando a nivel hardware con la aparición de arquitecturas con mayores posibilidades procesamiento y almacenamiento y a nivel software, con la aparición de modelos de programación que optimizan el procesamiento de los datos.

0.3. Los datos: los de ayer y los de hoy

Al igual que la tecnología ha ido evolucionando para dar respuesta a la ingente cantidad de datos que ha comenzado a generarse, estos últimos también han experimentado una gran evolución. Esta evolución, o revolución, no está únicamente relacionada con la **cantidad** de datos (como se expuso en el anterior apartado) sino también con el **tipo** y el **formato** de los mismos.

Tradicionalmente, el tipo y formato de datos con el que se ha trabajado para extraer información y conocimiento a partir de ellos era ciertamente **limitado**. En muchas ocasiones se trataba de ficheros de datos estructurados de forma tabular, donde cada fila del conjunto de datos representaba una instancia del mismo y cada columna una variable o atributo de la instancia. El formato de archivo que se manejaba solían ser **formatos de hojas de cálculo** (.xlsx, .ods, .numbers etc) o **ficheros separados por comas** (.csv). Muy pocos eran los procesos en los que se trabajaba con otros tipos de datos como texto, imágenes, audio e incluso vídeos, ya que los formatos de estos tipos de datos eran limitados hace unos años, su procesamiento más complejo y la tecnología para ello aún en desarrollo.

Aunque a día de hoy también se sigue trabajando con archivos de datos en forma de hojas de cálculo y/o archivos tradicionales para generar conocimiento a partir de ellos, las posibilidades actuales son prácticamente **ilimitadas**. En cuanto al texto, las técnicas de inteligencia artificial y procesamiento del lenguaje natural hacen posible la extracción de conocimiento a partir de **grandes volúmenes de textos**, que pueden provenir de páginas web, archivos .pdf, redes sociales, etc. El desarrollo de hardware con mejores prestaciones y los nuevos modelos de programación permiten procesar en la actualidad **grandes cantidades de imágenes, audios y vídeos** con una gran variedad de técnicas de inteligencia artificial en tiempos razonables. Finalmente, han aparecido nuevos tipos y formatos de datos, como por ejemplo, aquellos datos generados a partir de **grafos**, los cuales se tratarán en próximas secciones y capítulos con más detenimiento. Estos datos se corresponden, por ejemplo, con datos geográficos obtenidos a partir de mapas como los generados en aplicaciones como *Google Maps* u *Open Street Maps* o datos de seguimiento y actividad en redes sociales de gran valor en campañas publicitarias entre otros muchos (ver [HT14]).



Pregunta. Haz una búsqueda y elabora un listado con distintos tipos de datos y los formatos de almacenamiento más utilizados con los que se trabaja en ciencia de datos y big data.

Los diferentes tipos y formatos de datos, los de ayer y los de hoy, son la materia básica fundamental en cualquier proceso de extracción de información y de conocimiento. Después, las metodologías empleadas para ello y arquitecturas hardware sobre las que se realice el procesamiento de los mismos, permitirán definir **procesos y metodologías de big data**, aplicadas a un ámbito concreto.

0.4. Soluciones Big Data

En esta nueva era tecnológica en la que nos hayamos inmersos, a diario se generan enormes cantidades de datos, del orden de **petabytes** (más de un millón de gigabytes). Hoy en día, cualquier dispositivo como puede ser un reloj, un coche, un smartphone, etc está conectado a Internet generando, enviando y recibiendo una gran cantidad de datos. Tanto es así, que se estima que el 90 % de los datos disponibles en el mundo ha sido generado en los últimos años. Sin lugar a dudas, esta y las próximas generaciones serán las generaciones del **big data** (ver [HT14]).

Esta realidad descrita anteriormente demanda la capacidad de enviar y recibir datos e información a gran velocidad, así como la capacidad de almacenar tal cantidad de datos y procesarlos en tiempo real. Así pues, la gran cantidad de datos disponibles junto con las herramientas, tanto hardware como software, que existen a disposición para analizarlos se conoce como **big data**.

No existe una definición precisa del término **big data**, ni tampoco un término en castellano que permita denominar este concepto. A veces se usan en castellano los términos *datos masivos* o *grandes volúmenes de datos* para hacer referencia al big data. Por este motivo, a menudo el concepto de big data es definido en función de las características que poseen los datos y los procesos que forman parte de este nuevo paradigma de computación. Esto es lo que se conoce como las Vs del big data (ver [BNCD16]).

Algunos autores coinciden en que big data son datos cuyo **volumen** es demasiado grande como para procesarlos con las tecnologías y técnicas tradicionales, requiriendo nuevas arquitecturas hardware, modelos de programación y algoritmos para su procesamiento. Además, se trata de datos que se presentan en una gran **variedad** de estructuras y formatos: datos sintéticos, provenientes de sensores, numéricos, textuales, imágenes, audio, vídeo... Finalmente, se trata de datos que requieren ser procesados a gran **velocidad** para poder extraer valor y conocimiento de ellos. Esta concepción se conoce como las tres Vs del big data (ver figura 2).

Otros autores amplían las características que han de tener los datos que forman parte del big data, incluyendo “otras Vs” como lo son: **volatilidad**, referida al tiempo durante el cual los datos recogidos son válidos y a durante cuánto tiempo deberán ser almacenados. **Valor**, referido a la utilidad de los datos obtenidos para

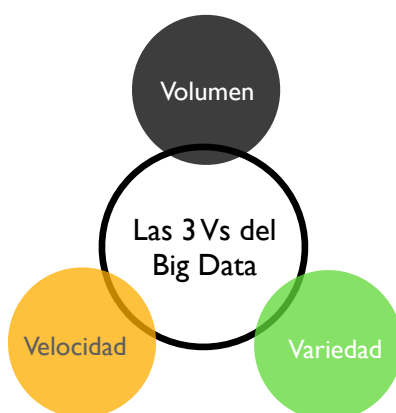


Figura 2: Definición de big data en base a “Las tres Vs del big data”.

extraer conocimiento y tomar decisiones a partir de ellos. **Validez**, referida a lo precisos que son los datos para el uso que se pretende darles. El uso de datos validados permitirá ahorrar tiempo en etapas como la limpieza y el preprocesamiento de los datos. **Veracidad**, relacionada con la confiabilidad del origen del cual provienen los datos con los que se trabajará así como la incertidumbre o el ruido que pudiera existir en ellos. **Variabilidad**, frente a la variedad de estructuras y formatos, hace referencia a la complejidad del conjunto de datos, es decir, al número de variables que contiene. Estas características, unidas a las tres Vs descritas anteriormente, se conocen como las ocho Vs del big data (ver figura 3).

Dado que no existe una definición uniforme para el término big data, muchos autores definen el término en función de aquellas características que consideran más relevantes, por lo que es común encontrar “las cinco Vs del big data”, “las siete Vs del big data” o “las diez Vs del big data” según cada autor, apareciendo distintos términos para describir el big data, como también pueden ser **visualización** o **vulnerabilidad**, entre otros.

Son muchas las **soluciones a nivel hardware y software**, que se han propuesto a los problemas derivados del almacenamiento y el procesamiento de big data. A continuación, se describen los fundamentos de tres de ellas, las cuales serán desarrolladas a nivel teórico, tecnológico y práctico en los siguientes capítulos.



Figura 3: Definición de big data en base a “Las ocho Vs del big data”.

0.4.1. Almacenes de datos

Las **bases de datos relacionales** son colecciones de datos integrados, almacenados en un soporte secundario no volátil y con redundancia controlada. La definición de los datos y la estructura de la base de datos debe estar basada en un modelo de datos que permita captar las interrelaciones y restricciones existentes en el dominio que se pretende modelizar (ver [dMP99]). A su vez, un **Sistema Gestor de Bases de Datos (SGBD)** se compone de una colección de datos estructurados e interrelacionados (una base de datos) así como de un conjunto de programas para acceder a dichos datos (ver [SKS06]).

Las bases de datos tradicionales, siguiendo la definición anterior, están basadas generalmente en sistemas relacionales u objeto-relacionales. Para el acceso, procesamiento y recuperación de los datos, se sigue el modelo **Online Transaction Processing (OLTP)**. Una transacción es una interacción completa con un sistema de base de datos, que representa una unidad de trabajo. Así pues, una transacción representa cualquier cambio que se produzca en una base de datos. El modelo OLTP, traducido al castellano como **procesamiento de transacciones en línea**, permite gestionar los cambios de la base de datos mediante la inserción, actualización y eliminación de información de la misma a través de transacciones básicas que son procesadas en tiempos muy pequeños. Con respecto a la recuperación de información de la base de datos, se utilizan operadores clásicos (concatenación, proyección, selección, agrupamiento...) para realizar consultas básicas y sencillas (realizadas, mayoritariamente, en lenguaje SQL y extensiones del mismo). Finalmente,

las opciones de visualización de los datos recuperados son limitadas, mostrándose fundamentalmente los resultados de forma tabular y requiriendo un procesamiento adicional y más complejo en caso de querer presentar datos complejos.

La revolución en la generación, almacenamiento y procesamiento de los datos, así como la irrupción del **big data**, han puesto a prueba el modelo de funcionamiento, rendimiento y escalabilidad de las bases de datos relacionales tradicionales. En la actualidad, se requiere de soluciones integradas que aúnen datos y tecnología para almacenar y procesar grandes cantidades de datos con diferentes estructuras y formatos con el objetivo de facilitar la consulta, el análisis y la toma de decisiones sobre los mismos. En este sentido, la **inteligencia de negocio**, más conocida por el término inglés **business intelligence**, investiga en el diseño y desarrollo de este tipo de soluciones. La inteligencia de negocio puede definirse como la capacidad de una empresa de estudiar sus acciones y comportamientos pasados para entender dónde ha estado la empresa, determinar la situación actual y predecir o cambiar lo que sucederá en el futuro, utilizando las soluciones tecnológicas más apropiadas para optimizar el proceso de toma de decisiones (ver [IGG03]).

Estas nuevas soluciones requerirán un modelo de procesamiento diferente a **OLTP**. Esto es así, ya que el objetivo perseguido por la inteligencia de negocio está menos orientado al ámbito transaccional y más enfocado al ámbito analítico. Las nuevas soluciones utilizan el modelo **Online analytical processing (OLAP)**. La principal diferencia entre OLTP y OLAP estriba en que mientras que el primero es un sistema de procesamiento de transacciones en línea, el segundo es un sistema de **recuperación y análisis** de datos en línea. Por tanto, OLAP complementa a SQL aportando la capacidad de analizar datos desde distintas variables y dimensiones, mejorando el proceso de toma de decisiones. Para ello, OLAP permite realizar cálculos y consolidaciones entre datos de distintas dimensiones, creando modelos que no presentan limitaciones conceptuales ni físicas, presentando y visualizando la información de forma flexible, esto es, en diferentes formatos. Los sistemas OLAP están basados, generalmente, en sistemas o interfaces multidimensionales que proporcionan facilidades para la transformación de los datos, permitiendo obtener nuevos datos más combinados y agregados que los obtenidos mediante las consultas simples realizadas por OLTP. Al contrario que en OLTP, las unidades de trabajo de OLAP son más complejas que en OLTP y consumen más tiempo. Finalmente, en cuanto a la visualización de los mismos, los sistemas OLAP permiten la visualización y el análisis multidimensional a partir de diferentes vistas de los datos, presentando los resultados en forma matricial y con mayores posibilidades estéticas y visuales. La tabla 1 muestra un resumen con las principales diferencias entre los sistemas **OLTP** y **OLAP**.

Un **almacén de datos**, más conocido por el término **data warehouse** (en inglés), es una solución de **business intelligence** que combina tecnologías y componentes con el objetivo de ayudar al uso estratégico de los datos por parte de una organización. Esta solución debe proveer a la empresa, de forma integrada, de capa-

Tabla 1: Tabla resumen y comparativa entre OLTP y OLAP

	Bases de datos relacionales	Soluciones Business Intelligence
	OLTP	OLAP
Concepto	Sistema de procesamiento de transacciones en línea	Sistema de recuperación y análisis de datos en línea
Funciones	Gestión de transacciones: inserción, actualización, eliminación...	Análisis de datos para dar soporte a la toma de decisiones
Procesamiento	Transacciones cortas	Procesamientos de análisis complejos
Tiempo	Las transacciones requieren poco tiempo de ejecución	Los análisis requieren mayor tiempo de ejecución
Consultas	Simples, utilizando operadores básicos tradicionales	Complejas, permitiendo analizar los datos desde múltiples dimensiones
Visualización	Básica. Muestra los datos en forma tabular	Muestra los datos en forma matricial. Mayores posibilidades gráficas

idad de almacenamiento de una gran cantidad de datos así como de herramientas de análisis de los mismos que, frente al procesamiento de transacciones, permita transformar los datos en información para ponerla a disposición de la organización y optimizar el proceso de toma de decisiones.

0.4.2. Bases de datos documentales u orientadas a documentos

La gran variedad y heterogeneidad en los tipos de datos almacenados y procesados en los últimos años ha puesto sobre la mesa la cuestión de si las bases de datos relacionales son el modelo más óptimo para trabajar con según qué tipos de datos. Como alternativa a ellas, en los últimos años han proliferado las bases de datos NoSQL.

NoSQL es el término utilizado para referirse a un tipo de bases de datos que permiten almacenar y gestionar tipos de datos que tradicionalmente han sido difíciles de gestionar por parte de las bases de datos relacionales. Así pues, NoSQL hace referencia a **bases de datos documentales, bases de datos orientadas a grafos, buscadores, etc.** En contraposición a las bases de datos relacionales, las NoSQL se caracterizan principalmente por:

- **Independencia del esquema:** Al contrario que en las bases de datos relacionales, no es necesario diseñar un esquema para definir los tipos y estructura de los datos almacenados, permitiendo acortar el tiempo de desarrollo y facilitando las modificaciones de la estructura interna de la base de datos.
- **No relacionales:** El concepto de relación de las bases de datos relacionales no existe en NoSQL. Por tanto, se trabaja con datos que no están normalizados, lo cual aporta flexibilidad en relación a los tipos y estructuras de datos que pueden ser almacenados.

- **Distribuidas:** La cantidad de datos almacenados requiere de su almacenamiento en múltiples servidores, ya que un único servidor por potente que sea no podrá procesar en tiempos razonables tal cantidad de información. Este hecho permite utilizar hardware sencillo, ya que al utilizar múltiples servidores no es necesario que todos ellos tengan grandes prestaciones.

Las **bases de datos documentales** trabajan con documentos, entendidos como una estructura jerárquica de datos que, a su vez, puede contener subestructuras. En una primera aproximación, es fácil pensar en que estos documentos pueden ser documentos de Microsoft Word, documentos pdf, páginas web... Las bases de datos documentales pueden, efectivamente, trabajar con estos tipos de documentos. Sin embargo, el término documento en este contexto posee un mayor nivel de abstracción.

Los **documentos** pueden consistir en datos binarios o texto plano. Es posible que se traten de datos semiestructurados, cuando aparecen en formatos como **JavaScript Object Notation (JSON)** o **Extensible Markup Language (XML)**. Por último, también pueden ser datos estructurados conforme a un modelo de datos particular como, por ejemplo, **XML Schema Definition (XSD)**.

Tabla 2: Tabla comparativa entre XML y JSON

	XML	JSON
Lenguaje fuente	SGML	JavaScript
Tipo Lenguaje	Orientado a datos	Orientado a documentos
Notación	Pesada	Ligera
Etiquetas inicio y fin	Sí	No
Comentarios	Sí	No
Espacios de nombres	Sí	No
Soporte tipos de datos	No	Sí

Actualmente, **XML** y **JSON** son los formatos de intercambio de datos más utilizados en el desarrollo de aplicaciones web. Sin embargo, existen importantes diferencias entre ellos: XML es una extensión del lenguaje **Standard Generalized Markup Language (SGML)** el cual es un lenguaje que permite la creación, organización y etiquetado de documentos. Por su parte, JSON es una extensión del lenguaje **JavaScript**. XML tiene una notación más pesada que JSON, siendo este último un lenguaje más ligero que admite tipos de datos y matrices. Por su parte, XML no proporciona tipos ni estructuras de datos, sino que contiene un conjunto de reglas que, mediante el uso de atributos y elementos, permite codificar un docu-

mento. Finalmente, JSON es un lenguaje orientado a datos mientras que XML es un lenguaje orientado a documentos. La tabla 2 muestra una comparativa de estos y otros aspectos de ambos lenguajes.

En **XML**, la estructura principal de un documento está formada por dos elementos: el **prólogo** (opcional) y el **cuerpo**. El prólogo contiene a su vez dos partes: la declaración XML que establece la versión del lenguaje, el tipo de codificación y si se trata de un documento autónomo y la declaración del tipo de documento. El cuerpo, por su parte, contiene la información del documento (ver [TR03]).

Supongamos que Carlos ha enviado un mensaje de whatsapp a Javier diciéndole que han quedado con los compañeros de trabajo a las diez de la noche en la puerta del Sol. Un **documento XML** que representa este mensaje como un documento, podría ser el mostrado en el listado 1.

Listado 1: Quedada en la puerta del sol

```
1 <?xml version="1.0" encoding="ISO-8859-1"?>
2 <whatsapp>
3   <para> Javier </para>
4   <de> Carlos </de>
5   <titulo> Quedada </titulo>
6   <contenido> A las 22:00 pm en la puerta del sol </contenido>
7 </whatsapp>
```

El listado 1 incluye en la primera línea el **prólogo** del documento, definiendo la versión y el tipo de codificación utilizada. A partir de la segunda línea, se define el **cuerpo** del documento que contiene, mediante etiquetas de apertura <> y cierre </>, los distintos atributos de los que se compone el documento.

En JSON, la sintaxis del lenguaje **tiene las mismas reglas que el lenguaje JavaScript**, del cual proviene. Sin embargo, los archivos JSON deben cumplir también otras reglas sintácticas adicionales. En primer lugar, un archivo JSON representará o bien un **objeto**, es decir, una tupla de pares clave-valor o bien una **colección de elementos**, es decir, un vector o array. Por otra parte, los archivos JSON que representan objetos comienzan con una llave de inicio { y terminan con una llave de cierre }. Cuando se representa un vector, sus elementos se encierran entre corchetes []. Las **cadenas** y **nombres de atributos** del objeto deberán encerrarse entre comillas, así como todos los nombres de los atributos del objeto, separándose cada elemento del siguiente con una coma (,) no habiendo una coma después del último elemento (ver [Fri19]).

Así pues, si se pretende representar en formato JSON el mensaje que ha enviado Carlos a Javier, el fichero JSON resultante sería el mostrado en el listado 2. Este fichero define un **objeto JSON** que contiene una serie de atributos entrecomillados cuyo valor asociado son cadenas de caracteres que, por tanto, también van entrecomilladas.

Listado 2: Quedada en la puerta del sol

```

1 {
2   "para": "Javier",
3   "de": "Carlos",
4   "titulo": "Quedada",
5   "contenido": "A las 22:00 pm en la puerta del sol"
6 }
```

Existen en la web múltiples aplicaciones online que permiten convertir archivos XML en JSON y viceversa. En sucesivos capítulos, se profundizará más sobre la sintaxis de estos dos lenguajes así como la equivalencia entre los mismos a la hora de definir documentos que serán tratados posteriormente por una base de datos documental. También se describirán las tecnologías más utilizadas en este ámbito, haciendo especial énfasis en **MongoDB**, uno de los sistemas de bases de datos NoSQL orientado a documentos más utilizado a día de hoy.

0.4.3. Bases de datos sobre grafos

Tal y como se expuso anteriormente, en los últimos años han aparecido nuevos tipos de datos como aquellos que vienen dados en forma de **grafo**. Algunos de estos datos son los provenientes de interacciones en **redes sociales**, **datos geográficos expresados en forma de mapas**, etc.

Un **grafo** es un ente matemático compuesto por un conjunto de **nodos o vértices** y un conjunto de **enlaces o aristas**. Matemáticamente puede ser expresado por medio de la ecuación (1).

$$G = \{V, E\} \quad (1)$$

Donde V representa el conjunto de nodos o vértices y E representa el conjunto de enlaces o aristas. Así pues, sea el grafo de la figura 4 que representa un conjunto de ciudades conectadas por autovías, el conjunto V de nodos sería $V = \{\text{La Coruña, Madrid, San Sebastián, Barcelona, Valencia, Sevilla, Cádiz}\}$, mientras que el conjunto de enlaces E vendría dado por $E = \{\text{A-1, A-2, A-3, A-4, A-4-I, A-4-II, A-6, A-7}\}$.

Una **base de datos orientada a grafos** es, por tanto, un sistema de bases de datos que implementa métodos de creación, lectura, actualización y eliminación de datos en un modelo expresado en forma de grafo. Existen dos aspectos fundamentales en este tipo de sistemas: el primero de ellos hace referencia al **almacenamiento de los datos**. En una base de datos orientada a grafos, los datos pueden almacenarse siguiendo el modelo relacional, lo que implica mapear la estructura del grafo a una

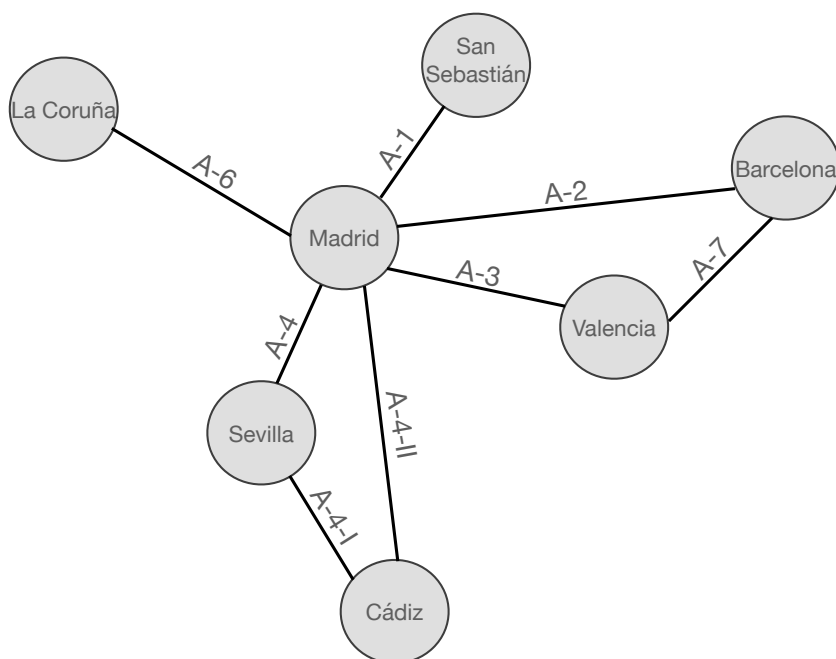


Figura 4: Grafo que representa las principales autopistas de España

estructura relacional, o bien, almacenarse de forma **nativa** utilizando modelos de datos propios para almacenar estructuras de tipo grafo. La ventaja de mapear los grafos a una estructura relacional radica en que la gestión y consulta de los datos se realizará de forma tradicional a través de un backend conocido como, por ejemplo, MySQL. Por su parte, la ventaja del almacenamiento nativo de grafos radica es que existen modelos de datos e implementaciones que aseguran y garantizan el buen rendimiento y la escalabilidad del sistema. El segundo aspecto importante es el **procesamiento de los datos**. En este sentido, también es posible distinguir el procesamiento **no nativo** de los grafos, lo cual implica el tratamiento de estos datos siguiendo las técnicas tradicionales utilizadas en los lenguajes de modificación de datos de las bases de datos relacionales, o el procesamiento **nativo** de los datos de grafos, el cual es beneficioso porque optimiza los recorridos del grafo cuando se realizan consultas aunque, en ocasiones, invierta demasiado tiempo y memoria en consultas que no requieren de recorridos complejos (ver [RWE15]).

Si bien es cierto que **cualquier dominio puede ser modelado como un grafo**, esta no es una razón de peso como para cambiar o migrar de un esquema y modelo de datos bien conocido e investigado, como el esquema relacional, a un modelo orientado a grafos. Sin embargo, la motivación para requerir de sistemas de bases de datos específicos, orientados a trabajar con este tipo de datos, radica en tres aspectos principales:

- **Rendimiento:** Los sistemas de bases de datos orientados a grafos optimizan el rendimiento de las consultas sobre este tipo de datos con respecto a las bases de datos relacionales. Si bien es cierto que las consultas en el modelo relacional se vuelven más complejas y el rendimiento disminuye a medida que el conjunto de datos crece, esto no es así **cuando se trabaja con grafos, donde complejidad y rendimiento permanecen constantes**. Esto es así, ya que las consultas se localizan en una porción del grafo y, por tanto, solo es necesario explorar la parte del grafo afectada para resolver la consulta.
- **Flexibilidad:** Los sistemas de bases de datos orientados a grafos son más flexibles que los sistemas de bases de datos relacionales. Mientras que los primeros requieren la modelización exhaustiva a priori de todo el dominio, esto no es necesario en los segundos, donde **la naturaleza aditiva de los grafos** permite añadir nuevos nodos, relaciones, etiquetas y subgrafos a uno dado **sin necesidad de modificar todo el modelo de datos**, facilitando la implementación y reduciendo el riesgo a corromper el modelo de datos.
- **Agilidad:** A partir de las dos características anteriores, es posible deducir que el trabajo con sistemas de bases de datos orientados a grafos es más ágil que la gestión de estos datos a través de sistemas de bases de datos relacionales. Esta rápida gestión permitirá utilizar tecnología alineada con las nuevas metodologías de **desarrollo de software ágil**, permitiendo diseñar y desarrollar software que utilice este tipos de datos.

Existen distintos lenguajes de marcado de grafos, a partir de los cuales es posible generar archivos con formato de grafo para ser tratados por una base de datos de este tipo. Algunos de los lenguajes más utilizados son **GraphML**, **eXtensible Graph Markup and Modeling Language (XGMML)**, **Graph Exchange Language (GXL)** o **Graph Modelling Language (GML)**, entre otros. La mayoría de ellos son variantes o extensiones del lenguaje XML para el modelado de grafos (ver [RB02]).

En la actualidad, **GraphML** (ver [BELP13]) es uno de los lenguajes más extendidos para el modelado de datos en forma de grafo. Se trata de un lenguaje sencillo, general, extensible y robusto que está compuesto por un núcleo del lenguaje que define las propiedades estructurales del grafo y un mecanismo flexible de extensiones que permite añadir información específica del mismo. La notación es muy similar a XML y se profundizará en ella en capítulos posteriores. A modo de ejemplo, el grafo mostrado en la figura 4 podría representarse en GraphML según se muestra en el listado 3.

Como se puede apreciar en el fragmento de código, en primer lugar se define un grafo cuyo identificador es Grafo_Autovías. El tipo de aristas que incluye este grafo son aristas **no dirigidas**, puesto que los enlaces del grafo no presentan flechas que indiquen dirección. Por tanto, los enlaces son bidireccionales. En caso de definir un grafo dirigido, el valor del atributo *edgedefault* sería *directed*. A continuación, se definen los distintos nodos que contendrá el grafo y después las aristas o enlaces,

definiendo el identificador de cada una de ellas así como su origen y destino. Al tratarse de un grafo no dirigido, origen y destino son intercambiables, no siendo así en caso de que se trate de un **grafo dirigido**.

Los sistemas de bases de datos orientados a grafos han proliferado mucho en los últimos años, existiendo numerosas tecnologías que dan soporte a ellos, como pueden ser **Microsoft Infinite Graph**, **Titan**, **OrientDB**, **FlockDB**, **AllegroGraph** o **Neo4j** que se estudiará con más profundidad posteriormente.

Listado 3: Grafo Autovías

```
1 <graph id="Grafo_Autovias" edgedefault="undirected">
2   <node id="La Coruña"/> <node id="San Sebastián"/>
3   <node id="Madrid"/>   <node id="Barcelona"/>
4   <node id="Valencia"/> <node id="Sevilla"/>
5   <node id="Cadiz"/>
6   <edge id = "A-6"      source="La Coruña" target="Madrid"/>
7   <edge id = "A-1"      source="Madrid"    target="San Sebastián"/>
8   <edge id = "A-2"      source="Madrid"    target="Barcelona"/>
9   <edge id = "A-7"      source="Barcelona" target="Valencia"/>
10  <edge id = "A-3"      source="Madrid"    target="Valencia"/>
11  <edge id = "A-4"      source="Madrid"    target="Sevilla"/>
12  <edge id = "A-4-I"    source="Sevilla"   target="Cádiz"/>
13  <edge id = "A-4-II"   source="Madrid"    target="Cádiz"/>
14 </graph>
```

Bibliografía

- [AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.
- [BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.
- [BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms*. Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.
- [dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.
- [FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.
- [Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.
- [GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.
- [HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.
- [IGG03] C. Imhoff, N. Gallemmo, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.

- [Inm02] William H. Inmon. *Building the Data Warehouse, 3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.
- [JVV03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.
- [KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.
- [MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.
- [MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.
- [PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma, 2006.
- [RB02] Pablo Rodríguez Bocca. *Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos*. Interoperabilidad, Monografía, 2002.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.
- [SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.
- [TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

Capítulo 2

Gestión de Soluciones - II

Julio Alberto López Gómez

En este capítulo se profundizará en los **almacenes de datos** o *Data Warehouses* (DW) como solución propia de la **inteligencia de negocio** o *Business Intelligence* (BI). Esta solución aparece como respuesta al crecimiento de datos digitales almacenados por las empresas y la necesidad de extraer información y generar conocimiento a partir de ellos para **optimizar el proceso de toma de decisiones** en cada uno de los departamentos o divisiones de la empresa. Para ello, en este capítulo se aborda el **concepto de almacén de datos, su arquitectura así como su diseño e implementación**.

2.1. Sistemas de ayuda a la decisión

En una empresa u organización, los datos generados a diario son, principalmente, aquellos derivados de las operaciones rutinarias de la empresa. Estos datos, tradicionalmente, se almacenaban en **bases de datos relacionales** y su manipulación se correspondía con **transacciones** realizadas sobre la base de datos. Sin embargo, el objetivo de cualquier organización es seleccionar esos datos para realizar estudios y análisis que permitan generar informes que, a su vez, permitan a la empresa **extraer información para tomar decisiones estratégicas** que conduzcan a la organización al éxito.

El crecimiento exponencial de los datos manejados por una organización ha hecho que los computadores sean las únicas herramientas capaces de procesar estos datos para obtener información y ofrecer ayuda en la toma de decisiones. En este contexto, aparecen los **sistemas de ayuda a la decisión** o *Decision Support Systems* (DSS) que ayudan a quienes ocupan puestos de gestión a tomar decisiones o elegir entre diferentes alternativas (ver [GR09]).



Sistema de ayuda a la decisión: Conjunto de técnicas y herramientas tecnológicas desarrolladas para procesar y analizar datos para ofrecer soporte en la toma decisiones a quienes ocupan puestos de gestión o dirección en una organización. Para ello, el sistema combina los recursos de los gestores junto con los recursos computacionales para optimizar el proceso de toma de decisiones.

Mientras que las bases de datos relacionales han sido tradicionalmente el componente del *back-end* en el diseño de sistemas de ayuda a la decisión, los almacenes de datos se han convertido en una opción mucho más competitiva como elemento *back-end* al mejorar el rendimiento de éstas.

Los **campos de aplicación** de los almacenes de datos no se reducen únicamente al ámbito empresarial, sino que cubren multitud de dominios como las **ciencias naturales, demografía, epidemiología o educación, entre otros muchos**. La propiedad común a todos estos campos y que hace de los almacenes de datos una adecuada solución en estos ámbitos es **la necesidad de almacenamiento y herramientas de análisis que permitan obtener en tiempos razonables información y conocimiento útiles para mejorar el proceso de toma de decisiones** (ver [GR09]).

2.2. Almacenes de datos: Concepto

La aparición de los **almacenes de datos** está ligada, principalmente, a una serie de **retos** que es necesario abordar para **convertir los datos transaccionales** con los que trabaja una base de datos relacional en **información para generar conocimiento y dar soporte al proceso de toma de decisiones** (ver [GR09]).

- **Accesibilidad:** Desde cualquier dispositivo, a cualquier tipo de usuario y a gran cantidad de información que no puede ser almacenada de forma centralizada. La accesibilidad, en este sentido, debe hacer frente al problema de la **escalabilidad** del sistema y de los datos que este maneja.
- **Integración:** Referente a la gestión de datos **heterogéneos**, con distintos formatos, y provenientes de distintos ámbitos de la organización. Una correcta integración debe garantizar a su vez la **corrección y completitud** de los datos integrados.
- **Consultas mejoradas:** Permitiendo incluir operadores avanzados y dar soporte a herramientas y procedimientos que posibiliten obtener el máximo partido de los datos existentes. De este modo, será posible obtener **información precisa para realizar un análisis eficiente**.
- **Representación multidimensional:** Proporciona herramientas para analizar de forma multi-dimensional los datos del sistema, incluyendo datos de **diferentes unidades** de la organización con el objetivo de proporcionar herramientas de **análisis y visualización multi-dimensional** para mejorar el proceso de toma de decisiones.

A continuación, se muestra una **definición de almacén de datos** muy extendida, dada por W. Inmon, quien es conocido por ser el “padre” del concepto de almacén de datos.



Almacén de datos (Data Warehouse): Colección de datos orientados a temas, integrados, variante en el tiempo y no volátil que da soporte al proceso de toma de decisiones de la dirección (ver [Inm02]).

Para entender correctamente esta definición, es necesario ahondar en las características que incluye la misma (ver [PVMCV06]).

- **Orientados a temas:** Es decir, no orientado a procesos (transacciones), sino a entidades de **mayor nivel de abstracción** como “artículo” o “pedido”.
- **Integrados:** Almacenados en un formato **uniforme y consistente**, lo que implica depurar o limpiar los datos para poder integrarlos.
- **Variante en el tiempo:** Asociados a un instante de tiempo (mes, trimestre, año...)
- **No volátiles:** Se trata de datos **persistentes** que no cambian una vez se incluyen en el almacén de datos.

El diseño y funcionamiento de los almacenes de datos se basa en el sistema de procesamiento analítico en-línea, **OLAP**. Este sistema se encarga del análisis, interpretación y toma de decisiones acerca del negocio, en contraposición a los sistemas de procesamiento de transacciones en línea, **OLTP**. Este sistema es encarnado por las bases de datos operacionales, las cuales ejecutan las transacciones de procesos operacionales del día a día.

Así pues, los sistemas **OLTP están dirigidos por la tecnología y orientados a automatizar las operaciones del día a día** de la organización, mientras que los sistemas **OLAP están dirigidos por el negocio y proporcionan herramientas para tomar decisiones a largo plazo**, mejorando la estrategia y la competitividad de la organización (ver [PVMCV06]). La tabla 2.1 muestra una comparativa entre las principales características de las bases de datos operacionales (OLTP) y los almacenes de datos (OLAP).

Tabla 2.1: Diferencias entre BBDD Operacionales y Almacenes de Datos (ver [GR09]).

Característica	BBDD Operacionales	Almacén Datos
<i>Objetivo</i>	Depende de la aplicación	Toma de decisiones
<i>Usuarios</i>	Miles	Cientos
<i>Trabaja con...</i>	Transacciones predefinidas	Consultas y análisis específicos
<i>Acceso</i>	Lectura y escritura a cientos de registros	Principalmente lectura. Miles de registros
<i>Datos</i>	Detallados, numéricos y alfanuméricos	Agregados, principalmente numéricos
<i>Integración</i>	En función de la aplicación	Basados en temas, con mayor nivel de abstracción
<i>Calidad</i>	Medida en términos de integridad	Medida en términos de consistencia
<i>Temporalidad Datos</i>	Solo datos actuales	Datos actuales e históricos
<i>Actualizaciones</i>	Continuas	Periódicas
<i>Modelo</i>	Normalizado	Desnormalizado, multidimensional
<i>Optimización</i>	Para acceso OLTP a parte de la BBDD	Para acceso OLAP a gran parte de la BBDD

2.3. Almacenes de datos: Arquitectura

Las arquitecturas disponibles para el **diseño de almacenes de datos** se basan, principalmente, en garantizar que el sistema cumpla una serie de propiedades esenciales para su óptimo funcionamiento (ver [JVVL03] y [GR09]).

- **Separación:** De los datos transaccionales y los datos estratégicos que sirven como punto de partida a la toma de decisiones.
- **Escalabilidad:** A nivel hardware y software, para actualizarse y garantizar el correcto funcionamiento del sistema a medida que el número de datos y usuarios aumenta.
- **Extensiones:** Permitiendo integrar e incluir nuevas aplicaciones sin necesidad de rediseñar el sistema completo.
- **Seguridad:** Monitorizando el acceso a los datos estratégicos guardados en el almacén de datos.



Las arquitecturas de almacenes de datos se clasifican, fundamentalmente, en dos tipos: arquitecturas **orientadas a la estructura** y arquitecturas **orientadas a la empresa**.

2.3.1. Arquitecturas orientadas a la estructura

Las arquitecturas orientadas a la estructura reciben su nombre debido a que están diseñadas poniendo especial énfasis en el **número de capas y elementos que componen la arquitectura del sistema de almacén de datos**. De acuerdo con este criterio, es posible distinguir las siguientes arquitecturas (ver [JVVL03],[MTK06],[GR09]).

Arquitectura de una capa

El objetivo principal de esta arquitectura, poco utilizada en la práctica, es **minimizar la cantidad de datos almacenados eliminando para ello los datos redundantes**. La figura 2.1 muestra un esquema de este tipo de arquitectura. En ella, **el almacén de datos creado es virtual**, existiendo un *middleware* que interpreta los datos operacionales y ofrece una vista multidimensional de ellos. **El principal inconveniente** de esta arquitectura es que su simplicidad hace que el sistema **no cumpla la propiedad de separación**, ya que los procesos de análisis se realizan sobre los datos operacionales.

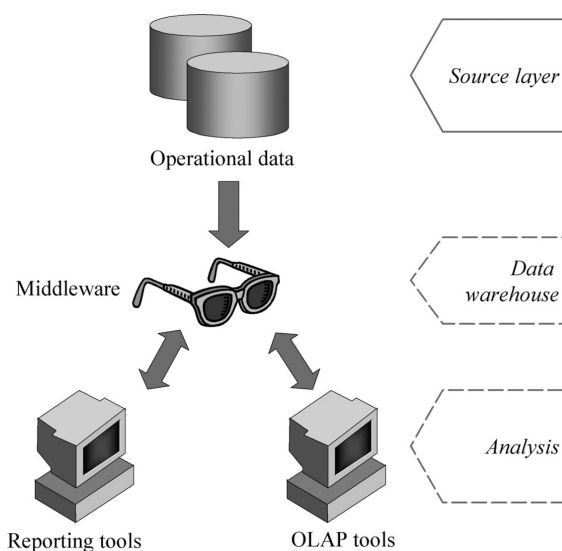


Figura 2.1: Almacén de datos. Arquitectura de una capa.

Arquitectura de dos capas

Fue diseñada con el objetivo de solucionar el problema de la separación que presentaba la arquitectura de una capa. Este esquema consigue **subrayar la separación entre los datos disponibles y el almacén de datos** a través de los siguientes componentes (ver figura 2.2):

- **Capa de origen (fuente):** Se corresponde con los orígenes y fuentes de los datos heterogéneos que se pretenden incorporar al almacén de datos.
- **Puesta a punto:** Proceso por el cual se utilizan herramientas de Extracción, Transformación y Carga (ETL) para extraer, limpiar, filtrar, validar y cargar datos en el almacén de datos.
- **Capa de almacén de datos:** Almacenamiento centralizado de la información en el almacén de datos, el cual puede ser utilizado para crear *data marts* o repositorios de metadatos.
- **Análisis:.** Conjunto de procesos a partir de los cuales los datos son eficientemente y flexiblemente analizados, generando informes y simulando escenarios hipotéticos para dar soporte a la toma de decisiones.



Un **Data mart** es un subconjunto o agregación de los datos almacenados en un almacén de datos primario que incluye información relevante sobre un área específica del negocio (ver [PVMCV06], [GR09]).

Arquitectura de tres capas

Este tercer tipo de arquitectura incluye una capa llamada de **datos reconciliados o almacén de datos operativos**. Con esta capa, los datos operativos obtenidos tras la limpieza y depuración son integrados y validados, proporcionando un modelo de datos de referencia para toda la organización. De este modo, **el almacén de datos no se nutre de los datos de origen directamente, sino de los datos reconciliados generados**, los cuales también son utilizados para realizar de forma más eficiente tareas operativas, como la realización de informes o la alimentación de datos a procesos operativos. La figura 2.3 muestra de forma gráfica este tipo de arquitectura. Esta capa de datos reconciliados también puede implementarse de forma virtual en una arquitectura de dos capas, ya que se define como una vista integrada y coherente de los datos de origen.

2.3.2. Arquitecturas orientadas a la empresa

Esta clasificación distingue **cinco tipos de arquitecturas** que combinan las capas mencionadas en la primera clasificación para diseñar almacenes de datos (ver [JVVL03], [GR09]).

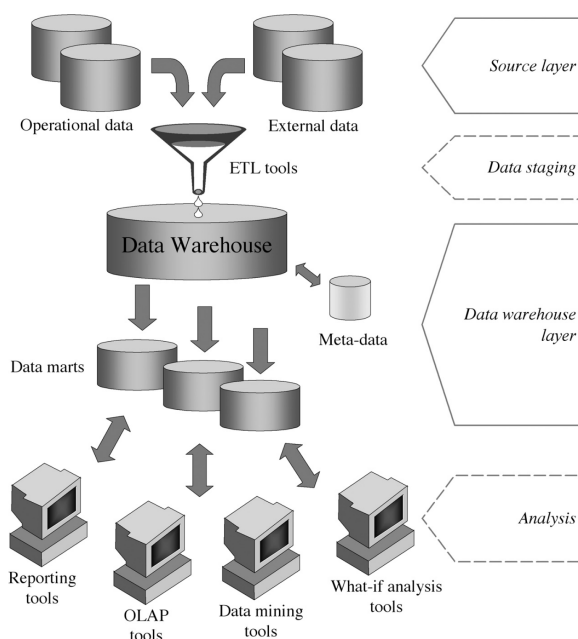


Figura 2.2: Almacén de datos. Arquitectura de dos capas.

Arquitectura de *data marts* independientes

Arquitectura preliminar en la que **los distintos *data marts* son diseñados de forma independiente y construidos de forma no integrada**. Suele utilizarse en los inicios de implementación de proyectos de almacenes de datos y reemplazada a medida que el proyecto va creciendo.

Arquitectura en bus

Similar a la anterior, **asegura la integración lógica de los *data marts* creados**, ofreciendo una visión amplia de los datos de la empresa y permitiendo realizar análisis rigurosos de los procesos que en ella se llevan a cabo.

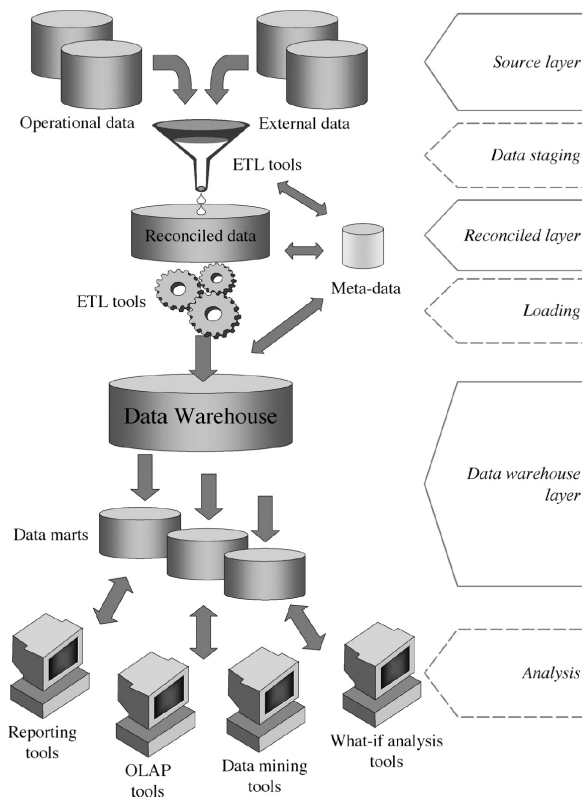


Figura 2.3: Almacén de datos. Arquitectura de tres capas.

Arquitectura hub-and-spoke (centro y radio)

Esta arquitectura es **muy utilizada en almacenes de datos de tamaños medio y grande**. Su diseño pone especial énfasis en garantizar la escalabilidad del sistema y permitir añadir extensiones al mismo. Para ello, **los datos se almacenan de forma atómica y normalizada en una capa de datos reconciliados que alimenta a los data marts** construidos que contienen, a su vez, los datos agregados de forma multidimensional. Los usuarios acceden a los *data marts*, si bien es cierto que también pueden hacer consultas directamente sobre los datos reconciliados.

Arquitectura centralizada

Se trata de un caso particular de la arquitectura hub-and-spoke. En ella, **la capa de datos reconciliados y los data marts se almacenan en un único repositorio físico**.

Arquitectura federada

Se trata de un tipo de arquitectura **muy utilizada en entornos dinámicos, cuando se pretende integrar almacenes de datos o *data marts* existentes con otros para ofrecer un entorno único e integrado de soporte a la toma de decisiones.** De esta forma, cada almacén de datos y cada *data mart* es integrado virtual o físicamente con lo demás. Para ello, se utilizan una serie de técnicas y herramientas avanzadas como son las ontologías, consultas distribuidas e interoperabilidad de metadatos, entre otras.

2.4. Almacenes de datos: Diseño e implementación

En esta sección se profundizará en cada una de las **etapas necesarias para diseñar e implementar un almacén de datos.** Para ello, en primer lugar se describirá el proceso de Extracción, Transformación y Carga (ETL), fundamental para construir y alimentar un almacén de datos. Después, se desarrollarán dos de los diseños más extendidos de almacén de datos: **el diseño en estrella y el diseño en copo de nieve.**

2.4.1. El proceso de Extracción, Transformación y Carga (ETL)

El proceso ETL es el encargado de **extraer, limpiar e integrar los datos provenientes de las fuentes de datos para alimentar el almacén de datos.** Este proceso también es el encargado de alimentar la capa de datos reconciliados en la arquitectura de tres capas. El proceso ETL **tiene lugar cuando se puebla el almacén de datos y se lleva a cabo cada vez que el almacén de datos se actualiza.** A continuación, se describen detalladamente cada una de las fases de las que consta este proceso (ver [KR02],[MTK06],[GR09]).

Extracción

Etapas que consiste en la **lectura de los datos de las distintas fuentes de las que provienen.** Cuando un almacén de datos se rellena por primera vez, se suele utilizar la técnica de **extracción estática**, la cual consiste en extraer una instantánea de los datos operacionales. A partir de entonces, se utiliza la **extracción incremental** para actualizar periódicamente los datos del almacén de datos, recogiendo los cambios aplicados desde la última extracción. Para ello, se utiliza el registro mantenido por el SGBD que, por ejemplo, asocia una **marca de tiempo (*timestamp*)** a los datos operacionales para registrar cuando fueron modificados y agilizar el proceso de extracción.

En la actualidad, existe una gran cantidad de conjuntos de datos o *data sets* públicos, conocidos bajo el nombre de *Open Data*, que abarcan una gran cantidad de dominios y con los que es posible trabajar para construir soluciones big data.



Open Data: Se trata de datos que han sido generados por una fuente en particular, que abarcan un dominio temático o disciplinar y tienen atributos, dentro de los cuales está la frecuencia de actualización. Además, cuentan con una licencia específica que indica las condiciones de reutilización de los mismos.

La fuente de los datos es en muchos de los casos el estado nacional, provincial, municipal u organizaciones comerciales. En otras ocasiones, **la fuente de los datos es fruto del estudio o medición por parte de particulares**. Los **atributos** de los conjuntos de datos deben especificar cómo fueron obtenidos, incluyendo fechas de obtención, actualización y validez, así como el público involucrado, la metodología de recogida o muestreo, etc.

Algunas de las fuentes más utilizadas en la actualidad para la obtención de datos abiertos provienen de **los centros nacionales e internacionales de estadísticas**, como son el Instituto Nacional de Estadística de España (INE)¹, eurostat², la oficina europea de estadísticas, la Organización Mundial de la Salud (OMS)³... Por otra parte, existen **repositorios públicos de datos abiertos y a disposición de los usuarios como UCI⁴ o Kaggle⁵**, que es una comunidad de científicos de datos donde empresas y organizaciones suben sus datos y plantean problemas que son resueltos por los miembros de la comunidad. La figura 2.4 muestra las fuentes de datos que se acaban de mencionar.



Figura 2.4: Fuentes para la obtención de datos abiertos.

¹<https://www.ine.es>

²<https://ec.europa.eu/eurostat>

³<https://www.who.int/es/data/gho/publications/world-health-statistics>

⁴<https://archive.ics.uci.edu/ml/index.php>

⁵<https://www.kaggle.com>

Transformación

La etapa de transformación es la fase clave para **transformar los datos operativos en datos con un formato específico para alimentar un almacén de datos**. En esta etapa, **los datos se limpian y se transforman, añadiéndoles contexto y significado**. En caso de implementar un almacén de datos siguiendo una arquitectura de tres capas, **el proceso de transformación es el encargado de obtener la capa de datos reconciliados**. Si bien es cierto que algunos autores separan la limpieza y la transformación de los datos en dos etapas distintas, en este capítulo se considerarán ambas dentro de la fase de transformación.



La etapa de **transformación** engloba todos los procesos de limpieza y manipulación de los datos, con el objetivo de transformar los datos operativos propios de sistemas relacionales (OLTP) en datos preparados para ser incluidos dentro del almacén de datos (OLAP).

La limpieza de los datos o *data cleaning* engloba todos aquellos procedimientos necesarios para detectar y resolver situaciones problemáticas con los datos de partida que pudieran suponer problemas potenciales a la hora de analizarlos. Así pues, los datos de partida pueden ser **incompletos**, es decir, pueden contener atributos sin valor o valores agregados, **incorrectos** que incluyan errores o valores sin ningún significado, lo cual es común cuando los datos se introducen manualmente en el sistema, o **inconsistentes** cuando los cambios no son propagados a todos los módulos del sistema, los rangos de un determinado atributo son cambiantes, existen datos duplicados...

Otra tarea muy importante que se debe abordar en la fase de limpieza es la gestión de los **datos perdidos**. Se trata de **datos que no están disponibles para algunas de las variables en alguna instancia**. Esto puede ser debido, por ejemplo, a que no se registraran las modificaciones sufridas por los datos, se produjera un mal funcionamiento del equipo, los datos nunca fueron rellenados o no existían en el momento en que fueron completados, se eliminaron por ser inconsistentes con la información registrada...

En general, se distinguen **dos tipos de situaciones cuando existen valores perdidos: datos perdidos completamente aleatorios y datos perdidos de forma no completamente aleatoria**. En el segundo caso, puede ser interesante intentar analizar la razón de la pérdida de los datos, la cual puede ser indicativa. En muchas ocasiones, los valores perdidos tienen relación con un subconjunto de variables predictoras y no se encuentran aleatoriamente distribuidos por todas ellas. Por todo ello, las aproximaciones más comunes a la hora de gestionar datos perdidos son:

- **Eliminación de instancias que contengan valores perdidos:** consiste en establecer un porcentaje umbral de tal forma que, si una instancia contiene un número de valores perdidos que supera este porcentaje, la instancia se descarta. Se trata de una técnica útil cuando el conjunto de datos es grande y el número de instancias con valores perdidos no es muy elevado. No obstante, esta estrategia se debe utilizar con precaución, ya que puede suponer una excesiva pérdida de información si el conjunto de datos no es muy grande, el número de instancias con valores perdidos es alto o el porcentaje umbral es muy pequeño.
- **Asignación de valores fijos:** consiste en asignar un valor fijo a todos los valores perdidos de todas las variables. Este valor puede ser el número 0 o incluso un valor desconocido *Unknown* o no numérico (NaN, Not a Number) en función del lenguaje de programación utilizado.
- **Asignación de valores de referencia:** asigna un valor de referencia a los valores perdidos para cada variable. Estos valores de referencia suelen ser medidas de centralización como la media o la mediana de los valores de cada variable.
- **Imputación de valores perdidos:** consiste en la aplicación de técnicas más sofisticadas, como pueden ser técnicas estadísticas o de aprendizaje automático para predecir o averiguar los valores que se han perdido.

Finalmente, la etapa de limpieza de datos también se encarga de la **detección de valores anómalos o outliers**. Se trata de valores que se han introducido de forma errónea o bien a una deformación en la distribución de valores. El proceso de detección de anomalías consiste, fundamentalmente, en dos etapas: en primer lugar, **asumir que existe un modelo generador de datos**, como podría ser una distribución de probabilidad. En segundo lugar, considerar que **las anomalías representan un modelo generador distinto**, que no coincide con el original. Existen multitud de técnicas para detectar y descartar o imputar valores anómalos, como lo son técnicas estadísticas basadas en la desviación y el rango intercuartílico o técnicas de aprendizaje automático.

Después de la etapa de limpieza, comienza la etapa de transformación. En ella, los datos se preparan para su carga en el almacén de datos. Para ello, se transformarán los datos provenientes de sistemas transaccionales OLTP en datos preparados para su análisis OLAP. A continuación, se muestran algunos de los procesos de transformación más comunes:

- **Estandarización de códigos y formatos de representación:** incluye tareas como transformar la información EBCDIC a formato ASCII o Unicode, convertir números cardinales en ordinales o viceversa, homogeneización y tratamiento de fechas, expandir codificaciones añadiendo textos descriptivos, unificación de códigos (sexo, estado civil, etc) y estándares (medidas, monedas, etc).

- **Conversiones y combinaciones de campos:** realización de agregaciones y cálculos simples (importe neto, beneficio, realización de cuentas y promedios), cálculos derivados con valores numéricos y textuales...
- **Correcciones:** en caso de que no se pudiera hacer en el origen o en la etapa de limpieza, se deben corregir errores tipográficos, datos sin sentido ni significado, resolver conflictos de dominio de variables, aclaración de datos ambiguos y asignación de valores a datos nulos.
- **Integración de varias fuentes:** implica la resolución de claves y utilización de diccionarios de datos y repositorios de metadatos para el diseño del almacén de datos.
- **Eliminación de datos y/o registros duplicados:** frecuente cuando se trabaja con distintas fuentes de datos provenientes de diferentes dimensiones o departamentos de la organización. Para ello, es muy útil el uso de funciones de búsqueda y agrupamiento aproximado.
- **Escalado y centrado:** para reducir los efectos adversos de contar con datos dispersos, es muy útil utilizar técnicas para escalar y centrar los datos. Una posible transformación consiste en restar a cada valor de cada instancia el valor medio de la variable correspondiente y después dividir los valores por la desviación estándar. De esta forma, los valores se escalan y se centran entorno al cero. Esta transformación también puede hacerse entorno al valor de la media o la mediana. También es posible escalar los valores en el intervalo $[0 - 1]$ o aplicar el logaritmo, para reducir la dispersión de los datos y mejorar la visualización.
- **Discretización:** la aplicación de muchas técnicas de aprendizaje automático requiere que los valores de las variables sean discreto, en lugar de continuos. La discretización es el proceso por el cual se divide un intervalo de valores continuos en tantos fragmentos como etiquetas o valores discretos se quieran asignar, sustituyendo los valores originales por la etiqueta o valor discreto correspondiente.
- **Selección de características:** permite reducir el coste computacional de trabajar con todas las variables del conjunto de datos cuando, en muchas ocasiones, existen variables que no aportan información para el aprendizaje. Además de reducir el coste computacional, se reduce el número de dimensiones del conjunto de datos, mejorando la visualización y mejorando el rendimiento de los modelos no solo en términos de tiempo, sino también de rendimiento, puesto que existen métodos de aprendizaje cuyo rendimiento es peor cuando se ejecutan utilizando variables no significativas.

Carga

Se trata de la última fase de cara a incluir datos en el almacén de datos. **La carga inicial de los datos puede requerir bastante tiempo al cargar de forma progresiva todos los datos históricos**, por lo que es normal realizarla en horas de baja carga de los sistemas. Una vez que el almacén de datos ha sido inicialmente cargado, las sucesivas cargas de datos se pueden realizar de dos formas (ver [GR09]):

- **Refresco:** El almacén de datos se reescribe completamente, de forma que se reemplazan los datos antiguos. El refresco **es utilizado habitualmente junto con la extracción estática** y suele ser una estrategia muy utilizada para **la carga inicial** del almacén de datos, aunque puede también realizarse a posteriori.
- **Actualización:** Se añaden al almacén de datos solamente aquellos datos nuevos que se pretenden incluir, sin eliminar ni modificar los datos ya existentes. Esta técnica es **utilizada frecuentemente junto con la extracción incremental** para actualizar regularmente el almacén de datos. Se trata de una estrategia de carga **compleja de diseñar**, ya que requiere que sea eficiente computacionalmente. Para ello, se requieren mecanismos de detección del cambio como aquellos basados en la marca de tiempo (*timestamp*) o la utilización de tablas de log, entre otros.

Frameworks ETL

Recientemente, han aparecido múltiples *frameworks* que ofrecen herramientas tecnológicas para dar un soporte integrado al proceso ETL. A continuación, se describen algunos de los más utilizados:

- **Bubbles (<http://bubbles.databrewery.org>):** Se trata de un *framework* ETL escrito en Python, aunque es posible utilizarlo desde otros lenguajes. Ofrece un marco de trabajo sencillo, donde el proceso ETL se describe como una secuencia de nodos conectados que representan los datos y las distintas operaciones que se pueden realizar con ellos. De esta forma, es posible construir un grafo que implemente el proceso ETL completo para un conjunto de datos.
- **Apache Camel (<https://camel.apache.org>):** Este *framework* escrito en lenguaje Java y de acceso abierto se enfoca en facilitar la integración entre distintas fuentes de datos, haciendo el proceso más accesible a los desarrolladores, ofreciendo distintas herramientas para dar soporte al proceso ETL.
- **Keettle (<https://community.hitachivantara.com/s/article/data-integration-kettle>):** Es el *framework* de Pentaho para dar soporte al proceso ETL. Pentaho es una *suite* o conjunto de programas para construir soluciones de inteligencia de negocio y Keettle es su entorno de trabajo ETL. Similar a Bubbles, ofrece un entorno de trabajo sencillo para implementar un proceso ETL a través de nodos y conexiones entre ellos. Además, Keettle es de acceso abierto.

2.4.2. Diseño en Estrella

A la hora de diseñar un almacén de datos, existen dos alternativas ampliamente utilizadas: el **diseño en estrella**, que promueve el diseño directo de estructuras lógicas sobre el modelo relacional, y el **diseño en copo de nieve**, como variante del diseño en estrella.

El proceso de desarrollo de un almacén de datos siguiendo el diseño en estrella puede estructurarse según las siguientes fases (ver [KR02],[PVMCV06]):

1. **Elegir un proceso de negocio a modelar:** cualquier proceso de negocio puede ser modelado como un almacén de datos. No obstante, serán de especial interés aquellos procesos **necesarios para la toma de decisiones y que involucran a diferentes unidades de la organización**.
2. **Escoger la granularidad del proceso de negocio:** Los datos almacenados en el almacén de datos **deben expresarse siempre al mismo nivel de detalle**. Por este motivo es necesario escoger un nivel de granularidad al comienzo del diseño del almacén de datos. En caso de que se pretenda almacenar información relevante con distintos niveles de granularidad, esta información deberá almacenarse de forma separada.
3. **Selección de medidas/hechos:** indican **qué se necesita medir o evaluar en el proceso de negocio** con el fin de dar respuesta a las necesidades de información y toma de decisiones por las cuales se pretende diseñar el almacén de datos. Por tanto, constituyen qué se quiere analizar. Las medidas/hechos son numéricas y es posible agregarlas a distintos niveles de detalle. Por ejemplo, en el ámbito logístico las medidas podrían ser las unidades aceptadas o devueltas, mientras que en el ámbito del control de calidad podrían ser las unidades producidas, defectuosas, costo de la producción.
4. **Elegir las dimensiones que se aplicarán a cada medida o hecho:** las dimensiones especifican **las distintas propiedades de las medidas o hechos que se pretenden almacenar e integrar dentro del almacén de datos**. Por ejemplo, si las medidas seleccionadas están relacionadas con las ventas de una determinada empresa, las dimensiones podrían ser: fecha de la venta, vendedor, cliente, producto...

A la hora de diseñar el almacén de datos siguiendo el diseño en estrella, se crea una **tabla central, también llamada tabla de hechos, tabla factual o fact table**. Esta tabla es la que posee los datos (medidas o hechos) sobre las diferentes combinaciones de las dimensiones. En algunas ocasiones, un diseño en estrella presenta más de una tabla de hechos. Los hechos introducidos pueden ser de distintos tipos: **completamente aditivos** (total de ventas de un departamento), **no aditivos** (márgen de beneficios expresado como porcentaje) o **semiaditivos** (como datos intermedios que pueden agregarse con datos de otras dimensiones).

Rodeando a la tabla de hechos, se encuentra **una tabla por cada una de las dimensiones definidas**. La clave primaria de la tabla de hechos se crea combinando las claves primarias de sus dimensiones relacionadas, de forma que está compuesta por las claves ajenas de las tablas de dimensiones que relaciona. De esta forma, la tabla de hechos presenta una relación del tipo “muchos a muchos” entre todas las tablas de dimensiones que relaciona, a la vez que una relación “muchos a uno” con cada tabla de dimensión por separado. Este esquema con forma de estrella da nombre a este tipo de diseño (ver [KR02],[MTK06]).

Sea un almacén de datos en el que se pretende almacenar **las ventas llevadas a cabo en una organización**. La tabla central de hechos representa los datos de cada venta, mientras que las dimensiones que rodean a la tabla central recogen datos sobre los productos vendidos, la fecha de la venta, el canal de distribución y el lugar de entrega. La figura 2.5 muestra un ejemplo de este esquema de almacén de datos.

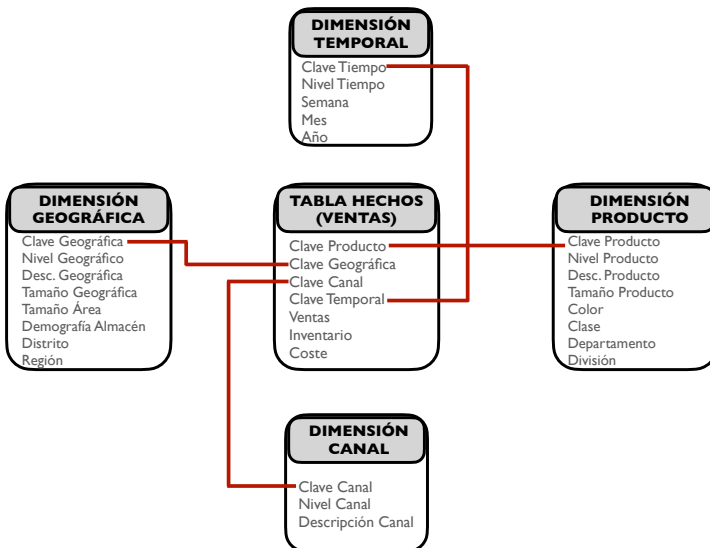


Figura 2.5: Ejemplo de almacén de datos diseñado en estrella.

Entre las **ventajas del diseño en estrella**, destaca ser un diseño **fácil de modificar que proporciona tiempos rápidos de respuesta**, simplificando la navegación de los metadatos. Además, este diseño **permite simular vistas de los datos que obtendrán los usuarios finales y facilita la interacción con otras herramientas**.

Sin embargo, el diseño en estrella presenta algunos **inconvenientes** que surgen, por ejemplo, cuando se **combinan dimensiones con distintos niveles de detalle**. Además, cuando se pretende limitar los niveles de una dimensión se debe añadir un campo de nivel o utilizar un modelo de tipo constelación, donde se incluyen diferentes estrellas que almacenan tablas de hechos agregadas, lo cual **añade complejidad al esquema**.



El diseño en estrella permite crear un almacén de datos con una estructura centralizada. El diseño se compone de una tabla de hechos central, que recoge los valores de las medidas del proceso de negocio que se pretende modelar. Rodeando a la tabla de hechos, se incluyen tantas tablas como dimensiones se hayan especificado en el modelo, las cuales se relacionan entre sí a través de la tabla de hechos.

2.4.3. Diseño en Copo de Nieve

Otro de los diseños más extendidos a la hora de implementar un almacén de datos es el **diseño en copo de nieve**. De hecho, el diseño de copo de nieve es considerado una variante o derivado del diseño en estrella y, por tanto, puede construirse a partir de este siguiendo las mismas etapas que se describieron en la sección anterior.

Este diseño es muy utilizado cuando se dispone de **tablas de dimensión con una gran cantidad de atributos**. En esta circunstancia, el diseño de copo de nieve permite **normalizar las tablas de dimensión**, ofreciendo un mejor rendimiento cuando las consultas realizadas sobre el almacén de datos requieren del uso de operadores de agregación. De esta forma, la tabla de hechos deja de ser la única tabla que presenta relaciones con las demás, apareciendo nuevas relaciones que se dan entre las tablas normalizadas que componen las tablas de dimensiones. Este esquema, que incluye ramificaciones en cada dimensión que se corresponden con las tablas necesarias para su normalización, guarda un gran parecido con la estructura de un copo de nieve, lo cual da nombre a este diseño (ver [KR02], [MTK06]).

La diferencia entre los diseños en estrella y copo de nieve radica fundamentalmente en la modelización de las tablas de dimensiones. Para obtener un diseño en copo de nieve, basta con partir de un diseño en estrella en el que, tras un proceso de normalización, se obtienen varias tablas de dimensión a partir de una tabla desnormalizada. Dentro del diseño de copo de nieve, es posible distinguir entre **copo de nieve parcial**, en el que solo algunas de las dimensiones son normalizadas y **copo de nieve completo**, en el que todas las dimensiones del esquema son normalizadas (ver [JVV03],[PVMCV06]).

Siguiendo con el ejemplo anterior, en el que se mostraba un almacén de datos para representar las ventas realizadas por una organización, la figura 2.6 muestra un diseño para un almacén de datos siguiendo un esquema de copo de nieve parcial, en el que las dimensiones geográfica y producto han sido normalizadas.

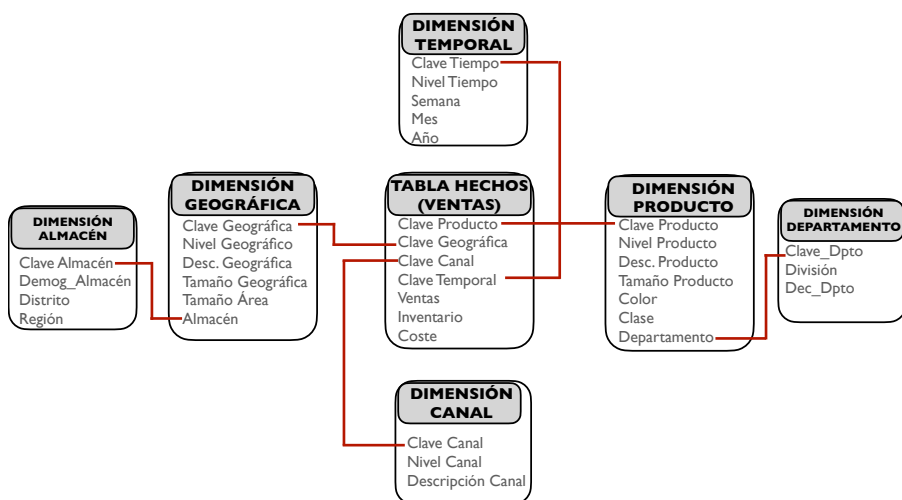


Figura 2.6: Ejemplo de almacén de datos diseñado en copo de nieve.

La normalización de las tablas de dimensiones proporciona al diseño de copo de nieve la **mejora de la eficiencia en la realización de consultas complejas que requieren el uso de operadores de agregación**. Sin embargo, este diseño es conceptualmente más difícil de implementar, ya que **incluye un mayor número de tablas** y requiere de realizar muchos *joins* entre ellas, lo que **incrementa los tiempos de consulta**.

En ocasiones, se requiere diseñar un almacén de datos que contenga más de una tabla de hechos. Para ello, **el esquema o diseño en constelación** permite crear un almacén de datos de forma similar al diseño en estrella, con la diferencia de que este modelo incluye más de una tabla de hechos. Además, cada tabla de hechos puede vincularse con todas o solo algunas tablas de dimensiones. Este esquema contribuye a la reutilización de las tablas de dimensiones, las cuales se pueden relacionar con más de una tabla de hechos.



El diseño en copo de nieve permite crear un almacén de datos a partir de un diseño en estrella. Para ello, las tablas de dimensiones se normalizan, generando más de una tabla para cada una de las dimensiones, mejorando la eficiencia de consultas complejas que requieren el uso de operadores avanzados.

Bibliografía

- [AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.
- [BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.
- [BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms*. Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.
- [dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.
- [FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.
- [Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.
- [GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.
- [HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

- [IGG03] C. Imhoff, N. Galemme, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.
- [Inm02] William H. Inmon. *Building the Data Warehouse, 3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.
- [JVV03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.
- [KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.
- [MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.
- [MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.
- [PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma, 2006.
- [RB02] Pablo Rodríguez Bocca. *Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos*. Interoperabilidad, Monografía, 2002.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.
- [SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.
- [TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

3

Capítulo

Gestión de Soluciones - III

Julio Alberto López Gómez

Este capítulo profundiza en dos soluciones de almacenamiento y gestión de datos propias de las bases de datos NoSQL: **las bases de datos XML y las bases de datos documentales**.

3.1. Bases de datos NoSQL

El término **NoSQL** proviene del inglés y está formado por las palabras “No” y “SQL”. El significado literal del término no debe confundir al lector ni alejarlo de su correcto significado. NoSQL no engloba solamente un conjunto de bases de datos y sistemas de almacenamiento que no utilizan SQL para su gestión. El significado de NoSQL amplía lo anterior, siendo el acrónimo de “*Not only SQL*”. Por tanto, NoSQL es una categoría general de sistemas de bases de datos que, además de SQL, utilizan otra serie de tecnologías para almacenar y gestionar los datos. El término NoSQL fue acuñado en 1998 por Carlo Strozzy y retomado en el año 2009 por Eric Evans, quien aprovechó este término para referirse a esta nueva generación de familias de bases de datos como “**Big Data**”. (ver [Tiw11]).

Las transacciones realizadas en los **sistemas de bases de datos relacionales** garantizan la consistencia y escalabilidad de las operaciones a través de una serie de características especificadas en el **modelo ACID**.

- **Atomicidad (A):** Garantiza que una transacción se ha realizado completamente o no, evitando que algunas operaciones se ejecuten a medias.
- **Consistencia (C):** También conocida como **integridad**, asegura que cualquier transacción realizada desde un estado seguro en la base de datos conducirá a esta a otro estado también seguro o válido.

- **Aislamiento (I: Isolation):** Permite que el resultado en el estado del sistema sea el mismo, independientemente del modo de ejecución de las transacciones (secuencial o concurrente).
- **Durabilidad (D): O persistencia,** garantiza que a pesar de que se produzca un fallo en el sistema, los cambios realizados por una transacción persistirán.

Alternativamente, las bases de datos **NoSQL** siguen el modelo conocido como **BASE**, más flexible que **ACID** en aquellos sistemas de bases de datos que no se adhieren estrictamente al modelo relacional.

- **Disponibilidad básica (Basic Availability):** El sistema siempre ofrece una respuesta a una petición de datos, aunque los datos sean inconsistentes, estén en fase de cambio o incluso se produzca un fallo.
- **Soft-state (S):** La consistencia de este tipo de bases de datos es eventual, por lo que el estado del sistema cambia a lo largo del tiempo, aunque no haya habido entradas de datos.
- **Eventual Consistency (E):** Cuando se dejan de recibir datos, el sistema se vuelve consistente. Así pues, los datos se propagan a todo el sistema, pero este continúa recibiendo datos sin evaluar la consistencia de los mismos para cada transacción antes de avanzar a la siguiente.

En la actualidad, las aplicaciones web modernas, el auge de la computación ubicua y el Big Data, presentan desafíos muy diferentes a los que presentan los sistemas de información tradicionales, implementados por medio de sistemas de bases de datos relacionales. Algunos **desafíos** son: el procesamiento masivo de datos, la alta frecuencia de lecturas y escrituras, los cambios dinámicos y frecuentes en el esquema de datos, la escalabilidad a costes razonables, gestión de datos temporales...En este contexto, en los últimos años han aparecido multitud de enfoques, modelos y tecnologías para aportar soluciones a los desafíos mencionados anteriormente. En este capítulo, se profundiza en **las bases de datos XML y las bases de datos documentales**.



Base de datos NoSQL: Sistema de bases de datos no relacional que utiliza distintas tecnologías para la gestión y almacenamiento distribuido de datos masivos sin un esquema estricto subyacente. Esta familia de base de datos permite incrementar la disponibilidad y escalabilidad del sistema, mejorando su rendimiento.

3.2. Bases de datos XML

El lenguaje **XML** (eXtensible Markup Language) es un lenguaje de marcas extensible derivado del lenguaje **SGML** (Standard Generalized Markup Language). Se trata de un lenguaje ideado para la definición y gestión de documentos que permite representar datos estructurados y semi-estructurados. En la actualidad, XML se ha convertido en un formato estándar para la comunicación entre aplicaciones, facilitando su integración. Entre las principales ventajas de XML destacan:

- **Datos autodocumentados:** Gracias a la posibilidad de definir cualquier tipo de etiquetas, no es necesario conocer el esquema para entender el significado de los datos.
- **Extensión:** Permite adaptar el lenguaje fácilmente a cualquier dominio.
- **Anidamiento:** La definición de estructuras anidadas dota a los documentos de gran flexibilidad para transferir y consultar información.
- **Flexibilidad:** Los documentos XML no tienen un formato estricto ni rígido, permitiendo añadir o ignorar información en ellos.

Estructura de un documento XML

La estructura de un documento XML está compuesta de elementos. Un **elemento** es una porción de documento delimitado por un par de etiquetas `<>` (apertura) y `< / >` (cierre) entre las cuales se incluye un texto llamado **contexto** del elemento. Un elemento sin contexto puede abreviarse mediante el uso de una única etiqueta `< nombre_elemento / >`. La estructura de un XML se forma **anidando** elementos para especificar la estructura del documento, formando una **estructura de árbol**. El fragmento de código 3.1 representa la estructura básica de un documento XML que representa un listado de asignaturas.

Listado 3.1: Listado de Asignaturas

```
1 <Asignaturas_Primer>
2   <Asignatura>
3     <titulo> Cálculo </titulo>
4     <tipo> Anual </tipo>
5     <profesor>
6       <nombre> Ana </nombre>
7       <apellido> Sanz </apellido>
8     </profesor>
9     <profesor>
10      <nombre> Roberto </nombre>
11      <apellido> Hernández </apellido>
12    </profesor>
13  </Asignatura>
14  <Asignatura>
15    <titulo> Física </titulo>
16    <tipo> Semestral </tipo>
17    <profesor>
```

```
18         <nombre> Carmelo </nombre>
19         <apellido> Moya </apellido>
20     </profesor>
21 </Asignatura>
22 </Asignaturas_Primer>
```

Los elementos de un documento pueden contener atributos. Los **atributos** se incluyen en la definición del elemento, dentro de la etiqueta de apertura. Sintácticamente, se representan mediante un par *nombre = valor*. A diferencia de un subelemento, **un atributo solo puede aparecer una vez en una etiqueta**.

El uso de atributos y su diferencia con respecto a los elementos no siempre es evidente. A **nivel de documento**, los atributos se introducen para añadir texto que no se visualizará en el documento, mientras que los subelementos formarán parte del contenido del documento. A **nivel de datos**, la diferencia es insustancial, puesto que la misma información se puede representar utilizando atributos o subelementos. Por ejemplo, el subelemento `< tipo >` de una asignatura, podría representarse como un atributo del elemento asignatura. El listado 3.2 representa la asignatura de Cálculo del ejemplo anterior, incluyendo el título de la asignatura como atributo de la misma.

Listado 3.2: Uso de atributos para la definición de asignaturas

```
1 <Asignaturas_Primer>
2   <Asignatura>
3     <titulo> Cálculo tipo = 'anual' </titulo>
4     <tipo> Anual </tipo>
5     <profesor>
6       <nombre> Ana </nombre>
7       <apellido> Sanz </apellido>
8     </profesor>
9   </Asignatura>
10  <Asignatura>
11    <nombre> Roberto </nombre>
12    <apellido> Hernández </apellido>
13  </Asignatura>
14 </Asignaturas_Primer>
```

Dado que una de las principales aplicaciones de XML es el intercambio de información entre organizaciones, y ante la flexibilidad del lenguaje que permite definir cualquier nombre para una etiqueta, es posible que **existan conflictos entre los nombres de éstas**. Para ello, se recurre a los **espacios de nombres**, los cuales permiten a las organizaciones especificar nombres de etiquetas únicos. De esta forma, se añade al nombre de la etiqueta o atributo un identificador de recursos universal. El listado muestra de forma resumida un ejemplo de utilización del espacio de nombres para la definición de asignaturas.

Listado 3.3: Uso de un espacio de nombres

```
1 <Asignatura xmlns:AS="http://www.listado-asignaturas.com">
2   <AS:titulo> Cálculo tipo = 'anual' </AS:titulo>
3   <AS:tipo> Anual </AS:tipo>
```

```
4      <AS:profesor>
5          <AS:nombre> Ana </AS:nombre>
6          <AS:apellido> Sanz </AS:apellido>
7      </AS:profesor>
8      <AS:profesor>
9          <AS:nombre> Roberto </AS:nombre>
10         <AS:apellido> Hernández </AS:apellido>
11     </AS:profesor>
12 </Asignatura>
```

Al igual que ocurre en las bases de datos relacionales, las bases de datos XML presentan una serie de **lenguajes de definición de tipos de documentos XML**. Concretamente, a continuación se profundizarán en **DTD** y **XML Schema**. Seguidamente, se estudiarán los lenguajes **XPath**, **XSLT** y **XQuery**, como **lenguajes de manipulación de datos XML** para la definición y ejecución de consultas.

3.2.1. Definition Type Document (DTD)

En XML, se dice que un documento está **bien formado** si cumple con la sintaxis del lenguaje: correcta definición de elementos y atributos, correcto anidamiento... Como se puede apreciar, este concepto alude a la **dimensión sintáctica de los documentos**.

Por otra parte, se dice que un documento XML es **válido** cuando **está bien formado y cumple con la estructura de documento que ha sido dada a través de un lenguaje de definición de datos**. Esta definición de la estructura de los documentos XML es **opcional**, si bien es cierto que es recomendable, especialmente cuando se trabaja con documentos complejos y con un volumen grande de ellos. La definición de tipos de documento o **DTD permite definir la estructura de un documento XML**. Para ello, la DTD especifica la estructura de los elementos y atributos de un documento: qué elementos pueden aparecer, qué atributos, qué subelementos, multiplicidad de ellos... Existen multitud de validadores on-line de documentos XML y sus correspondientes DTDs y/o XML Schemas¹.

La **declaración de una DTD** dentro de un documento XML puede implementarse de dos formas: en primer lugar, de manera **interna**. En este caso, se añade la DTD después del preámbulo XML y justo antes del documento propiamente dicho. El listado 3.4 muestra un ejemplo de declaración interna de una DTD.

Listado 3.4: Declaración Interna DTD

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2 <!DOCTYPE Elemento_Raiz [Aquí la DTD]>
3 <!--Aquí va el documento XML-->
```

¹<https://www.xmlvalidation.com>

<http://xmlvalidator.new-studio.org>

En segundo lugar, la **DTD puede declararse de forma externa**, definiendo la misma en un fichero .DTD aparte que después es enlazado en el documento. El listado 3.5 muestra un ejemplo de declaración externa de una DTD.

Listado 3.5: Declaración Externa DTD

```
1 <?xml version="1.0" encoding="UTF-8" standalone="yes" ?>
2 <!DOCTYPE Elemento_Raiz SYSTEM "Mi_DTD.dtd">
3 <!-------Aquí va el documento XML----->
```

Siguiendo con el ejemplo del listado 3.1, el listado 3.6 muestra el archivo .dtd que validaría la estructura del documento de asignaturas.

Listado 3.6: DTD Listado de Asignaturas

```
1 <! DOCTYPE Listado_Asignaturas[
2 <! ELEMENT Asignaturas_Primeros (Asignatura+)>
3 <! ELEMENT Asignatura (titulo, tipo, profesor+)>
4 <! ELEMENT titulo (#PCDATA)>
5 <! ELEMENT tipo (#PCDATA)>
6 <! ELEMENT profesor (nombre, apellido)>
7 <! ELEMENT nombre (#PCDATA)>
8 <! ELEMENT apellido (#PCDATA)>
9 ]>
```

En el listado anterior, se define en primer lugar el tipo de documento “Listado_Asignaturas”. A continuación, y especificados entre corchetes, se describen los elementos que contendrá el documento, comenzando por el elemento raíz “Asignaturas_Primeros”. Este elemento estará formado por uno o más elementos “Asignatura” (especificado mediante el operador +). Seguidamente, el elemento “Asignatura” viene dado por un conjunto de tres subelementos: título, tipo y profesor. Además, cada asignatura puede tener uno o más profesores. Los elementos título y tipo vienen dados por cadenas de caracteres (denotadas con la instrucción #PCDATA) mientras que el elemento profesor estará formado por dos subelementos llamados “nombre” y “apellido” ambos definidos mediante cadenas de caracteres.

La definición de atributos para un elemento se hace a continuación de la definición del mismo utilizando `<!ATTLIST >`. El listado 3.7 muestra la DTD anterior en caso de que el tipo de asignatura sea considerado un atributo.

Listado 3.7: DTD Listado de Asignaturas con atributos

```
1 <! DOCTYPE Listado_Asignaturas[
2 <! ELEMENT Asignaturas_Primeros (Asignatura+)>
3 <! ELEMENT Asignatura (titulo, profesor+)>
4 <! ATTLIST tipo #CDATA REQUIRED>
5 <! ELEMENT titulo (#PCDATA)>
6 <! ELEMENT tipo (#PCDATA)>
7 <! ELEMENT profesor (nombre, apellido)>
8 <! ELEMENT nombre (#PCDATA)>
```

```
9  <! ELEMENT apellido (#PCDATA)>
10 ]>
```

Finalmente, la tabla 3.1 muestra un resumen de definición de elementos y atributos en una DTD.

Tabla 3.1: Descripción DTD de elementos y atributos XML

ELEMENTO			ATRIBUTO		
Tipos	#PCDATA	Cadena de caracteres	Tipos	CDATA	Cadena de caracteres
	EMPTY	Elemento sin contexto		ID	Identificador
	ANY	Cualquier tipo		IDREF	Referencia a un ID
		Disyunción		IDREFS	Referencia a múltiples ID.
Operadores	+	Uno o más	Valores	#REQUIRED	Obligatorio
	*	Cero o más		#IMPLIED	Opcional
	?	Cero o uno		#FIXED	Valor fijo
			#DEFAULT		
			Valor por defecto		

3.2.2. XML Schema

Se trata de un **lenguaje de definición de la estructura de documentos XML más sofisticado que DTD**. Aunque es **más complejo, soluciona muchos de los inconvenientes de las DTD**. De hecho, XML Schema es un superconjunto del lenguaje DTD especificado mediante la sintaxis de XML. Entre otros aspectos, XML Schema soporta no solo el tipo cadena de caracteres (soportado por DTD), sino también tipos numéricos, tipos complejos como listas y además permite al usuario definir sus propios tipos así como extender tipos de datos complejos mediante un mecanismo similar a la herencia. Además, XML Schema soporta el concepto de espacios de nombre, permitiendo reutilizar elementos de un esquema en otros.

Un XML Schema se especifica en un documento aparte del documento XML con extensión **.xsd**. Dentro de este archivo, que sigue las reglas sintácticas de cualquier documento XML, se especifican las definiciones de cada elemento del esquema, qué atributos incluye así como sus tipos de datos válidos asociados tanto a los elementos como a los atributos. De esta forma, **XML Schema permite definir un esquema para validar documentos XML, de forma más sofisticada a como lo hace DTD**. El listado 3.8 muestra un XML Schema para el ejemplo del listado de asinaturas.

Listado 3.8: XML Schema: Listado de asignaturas

```

1 <?xml version="1.0" encoding="utf-8"?>
2 <xs:schema attributeFormDefault="unqualified" elementFormDefault="qualified" xmlns:xs="
  http://www.w3.org/2001/XMLSchema">
3   <xs:element name="Asignaturas_Primer">
4     <xs:complexType>
5       <xs:sequence>
6         <xs:element maxOccurs="unbounded" name="Asignatura">
7           <xs:complexType>
8             <xs:sequence>
9               <xs:element name="titulo" type="xs:string" />
10              <xs:element name="tipo" type="xs:string" />
11              <xs:element maxOccurs="unbounded" name="profesor">
12                <xs:complexType>
13                  <xs:sequence>
14                    <xs:element name="nombre" type="xs:string" />
15                    <xs:element name="apellido" type="xs:string" />
16                  </xs:sequence>
17                </xs:complexType>
18              </xs:element>
19            </xs:sequence>
20          </xs:complexType>
21        </xs:element>
22      </xs:sequence>
23    </xs:complexType>
24  </xs:element>
25 </xs:schema>

```

El listado anterior comienza definiendo un XML Schema en la segunda línea de código. Cualquier XML Schema deberá incluir esta línea de código. El atributo *attributeFormDefault* = “*unqualified*” indica que no es necesario utilizar el prefijo del espacio de nombres para definir los atributos del esquema, mientras que el atributo *elementFormDefault* = “*qualified*” indica que no es necesario utilizar el prefijo del espacio de nombre para definir los elementos del esquema. A continuación, se define el elemento raíz “Asignaturas_Primer” el cual se trata de un tipo complejo, puesto que no se define con un tipo de datos básico como entero, carácter, etc. Este elemento está formado por una secuencia de elementos del tipo “Asignatura” (el número máximo de elementos Asignatura no está limitado ya que *maxOccurs* = “*unbounded*”). Los elementos Asignatura, a su vez, están formados de una secuencia de elementos “nombre”, “tipo” (ambos de tipo cadena de caracteres) y un conjunto de elementos de tipo “profesor” no acotados, el cual se compone a su vez de los elementos “nombre” y “apellido”, ambos de tipo cadena de caracteres.

XML Schema permite representar la cardinalidad de un elemento, definiendo el número mínimo y máximo de ocurrencias mediante los atributos *minOccurs* y *maxOccurs*. Además, también es posible añadir restricciones de forma que se limiten los valores de un elemento o atributo, se permita elegir entre una lista de posibles... Finalmente, XML Schema dispone de los contenedores *sequence* para definir una secuencia ordenada de subelementos, *choice* para definir la elección entre posibles grupos de elementos o elementos y *all* para definir un conjunto no

ordenado de elementos. Al igual que sucedía con DTD, **existen distintas herramientas on-line que no solo permiten validar un documento XML a partir de su esquema XSD, sino que también permiten generar el esquema XSD a partir del archivo XML**².

3.2.3. XPath

En cualquier sistema de bases de datos, además de los lenguajes de definición de datos, los **lenguajes de manipulación de datos** son esenciales para poder **realizar consultas** y recuperar datos, realizar actualizaciones, etc.

XPath es un lenguaje de manipulación de datos XML. La principal característica de este lenguaje es que **interpreta un documento XML como una estructura de datos de tipo árbol**, donde los nodos y sus respectivas hojas se corresponden con los elementos y subelementos del documento XML. De esta forma, y utilizando comandos que permiten recorrer el documento de forma análoga a los comandos que se utilizan en una terminal de comandos para recorrer el sistema de archivos de un computador, es posible realizar consultas. La tabla 3.2 muestra un listado de los principales operadores utilizados en XPath.

Tabla 3.2: XPath: Operadores básicos

Operador	Significado
/	Navegación entre nodos del documento. Selecciona los nodos del nivel inferior
//	Navegación entre nodos del documento. Selecciona los nodos del nivel inferior de cualquier nodo especificado a continuación
.	Selecciona el nodo actual
..	Selecciona el nodo padre o nodo de nivel superior
	Expresa alternativas
@	Selecciona un atributo de un nodo
*	Selecciona todos los nodos
[]	Agrupar otros operadores

Con estos operadores, veamos cómo realizar algunas consultas de ejemplo sobre el fragmento XML que especifica un listado de asignaturas.

²<https://www.freeformatter.com>
<https://www.liquid-technologies.com/online-xsd-validator>,<https://extendsclass.com/xml-schema-validator.html>

Consulta 1. Recuperar los nombres de todas las asignaturas



```
/Asignaturas_Primer/Asignatura/titulo/text()
```

Consulta 2. ¿Cuántas asignaturas son impartidas por más de un profesor?



```
/Asignaturas_Primer/Asignatura[count(profesor)>1]
```

Consulta 3. ¿Quiénes son los profesores que imparten la asignatura de física?



```
/Asignaturas_Primer/Asignatura[titulo="Física"]/profesor/nombre
```

Es posible utilizar XPath de forma on-line a través de visualizadores³ o, por ejemplo, instalando el plug-in XPatherizer de Notepad++, en caso de utilizar este último editor de código XML.

3.2.4. XSLT

Otro lenguaje de manipulación de datos XML es el lenguaje XSLT. **El lenguaje XSLT (XML Stylesheet Transformations) proviene del lenguaje XSL (XML Stylesheet).** Este último permite especificar las opciones de formato de un documento XML en un archivo separado, aislando así la especificación del contenido y la presentación de un archivo XML. Podría decirse que XSL es a XML lo que CSS es a HTML. **El lenguaje XSLT permite especificar las opciones e formato de un documento XML, pudiendo transformar este documento en otros formatos como HTML, PDF, etc.** Dada la potencia de este lenguaje, que es capaz de transformar un archivo XML en otro que también pudiera ser XML, **se utiliza frecuentemente como lenguaje de consulta**

Las transformaciones XSLT se definen por medio de **plantillas**, que permiten a su vez recuperar contenido del documento XML a través de la utilización de expresiones XPath. La principal característica de XSLT es que la aplicación de plantillas se realiza de forma recursiva, lo cual recibe el nombre de **recursividad estructural**. El listado 3.9 presenta un ejemplo de aplicación de una transformación XSLT por medio de una plantilla.

³https://download.cnet.com/XPath-Visualizer/3000-7241_4-75804649.html,
<https://www.freeformatter.com/xpath-tester.html>

Listado 3.9: XSLT: Ejemplo de plantilla

```
1 <xsl: template match = /Asignaturas_Primer/Asignatura >
2 <titulo_asignatura>
3   <xsl:value-of select = titulo/>
4 </titulo_asignatura>
5 </xsl:template>
```

Tal y como se puede observar en el fragmento anterior, una transformación o consulta XSLT se define mediante dos órdenes básicas: *match* que especifica una expresión XPath para seleccionar uno o más nodos y *value – of select* que devuelve los valores especificados de los nodos que se han obtenido como resultado de la expresión XPath. Merece la pena destacar que **cualquier texto o etiqueta del archivo XSLT que no esté en el espacio de nombre se copia a la salida sin cambios**. Esto quiere decir, que el resultado del fragmento anterior no son únicamente los títulos de las asignaturas, sino estos mismos delimitados por una etiqueta de apertura y cierre llamada `< titulo_asignatura >`. De esta forma, a partir de un documento XML se ha generado otro con el resultado de la consulta. A modo de resumen, la tabla 3.3 define los principales constructores XSLT.

Tabla 3.3: Principales Constructores XSLT

Constructor XSLT	Significado
<code>< xsl : template match = "expresion XPath" ></code>	Selecciona nodos a devolver
<code>< xsl : value – of select = "valor" ></code>	Devuelve valores de los nodos seleccionados
<code>< /xsl : template >< xsl : template match = "*" / ></code>	Devuelve todos los nodos que no coinciden con alguna otra plantilla
<code>< xsl : attribute nombre = "nombre atributo" ></code>	Añade un atributo al elemento precedente
<code>< xsl : apply – templates/ ></code>	Aplica la recursividad estructural
<code>< xsl : keyname = "nombre clave" match = "elementos aplicar" use = "valor clave" / ></code>	Define una clave
<code>< xsl : value – of select = "key("nombre clave" "valor")"</code>	Devuelve los nodos que coincidan con el valor especificado. Especifica operaciones de combinación (JOINS)
<code>< xsl : sort select = "elemento o atributo" / ></code>	Devuelve los nodos resultados, ordenados por un elemento o atributo

Para trabajar con XSLT, se aconseja la descarga del software XML Copy Editor (<https://xml-copy-editor.sourceforge.io>).

3.2.5. XQuery

Es un **lenguaje de consulta de propósito general sobre datos XML estandarizado por el consorcio W3C**. XQuery procede del lenguaje de programación Quilt y, por este motivo, toma prestadas muchas características de otros lenguajes como XPath o SQL. De hecho, la sintaxis de XQuery es mucho más similar a la de SQL, al contrario que ocurría con lenguajes como XPath o XSLT.

Para definir una consulta en XQuery se construye una **expresión, denominada “FLWR”(flower, flor en inglés)**. A continuación, se describen cada una de las sentencias de las que se compone la expresión:

- **FOR:** Obtiene una serie de variables cuyos valores son el resultado de una expresión XPath. Es el equivalente a la cláusula **FROM** del lenguaje SQL.
- **LET:** Define y renombra expresiones complejas para ser utilizadas en el resto de la consulta, reduciendo la complejidad y aliviando la sintaxis. Es una cláusula **opcional**.
- **WHERE:** En esta cláusula, llamada igualmente en SQL, se especifican condiciones sobre los resultados obtenidos en la cláusula FOR.
- **RETURN:** Especifica y define el resultado que se va a obtener de la consulta. Se trata del equivalente a la cláusula **SELECT** en SQL.

Siguiendo la sintaxis anteriormente mencionada, el listado 3.10 muestra un ejemplo para obtener un listado con las asignaturas anuales.

Listado 3.10: XQuery: Ejemplo de consulta

```
1 for $x in /Asignaturas_Primer/Asignatura
2 let $nombre_asignatura:= $x/titulo
3 where $x/tipo='anual'
4 return <asignatura_anual> $nombre_asignatura </asignatura_anual>
```

En el listado anterior, la cláusula *for* permitirá recorrer todos los elementos asignatura. Por su parte, *let* define una expresión en la que se asigna a *nombre_asignatura* el título de la asignatura que se está recorriendo en cada momento. La cláusula *where* impone el criterio de que el tipo de asignatura sea anual y, finalmente, *result* define como resultado un fragmento de código XML donde el nombre de la asignatura anual (según la expresión indicada en *let*) aparecerá como resultado encerrado entre dos etiquetas XML llamadas *asignatura_anual*. Tal y como se puede apreciar, XQuery también permite generar nuevos documentos XML a partir de consultas.

Al igual que SQL, XQuery dispone de multitud de funciones para la consulta de datos. Así, es posible utilizar la función *distinct()* para eliminar duplicados en los resultados así como funciones de agregación como *sum()* o *count()* además de poder especificar funciones definidas por el usuario. XQuery también permite

utilizar “joins” incluyendo en la cláusula *where* el operador de combinación al igual que en SQL. Por otra parte, si bien es cierto que XQuery no posee el operador *group by* de SQL, este puede implementarse mediante la creación de consultas anidadas, las cuales sí que son soportadas por el lenguaje.

3.3. Bases de datos documentales: MongoDB

Los sistemas de bases de datos documentales u orientados a documentos son **sistemas de bases de datos NoSQL que almacenan datos, los cuales se estructuran en forma de documentos**. En los últimos años, han proliferado una gran variedad de sistemas de este tipo como pueden ser **Cassandra⁴, CouchDB⁵, Riak⁶ o MongoDB⁷**, sobre el cual trata esta sección.

El nombre **MongoDB** proviene de la palabra inglesa “**homongous**” que significa enorme. **MongoDB es, por tanto, un sistema de base de datos NoSQL, open source, orientado a documentos y escrito en lenguaje C++**. Este sistema de bases de datos está disponible no solo para múltiples plataformas y sistemas operativos (Windows, Linux, OS X) sino también como servicio empresarial en la nube y se puede integrar con otros servicios como, por ejemplo, Amazon Web Services (AWS).

3.3.1. MongoDB: Características y aplicaciones

Tal y como se ha presentado anteriormente, MongoDB es un sistema de bases de datos documental u orientado a documentos. Esta orientación hace que los datos se almacenen de forma estructurada en forma de documentos, los cuales se acoplan sin problema en los tipos de datos utilizados por los lenguajes de programación. Además, esta concepción hace que una base de datos MongoDB disponga de un esquema dinámico y fácilmente modificable.

En este apartado se pretende poner de relieve **las principales características de MongoDB** que hacen que sea en la actualidad uno de los sistemas de bases de datos NoSQL más utilizados tanto en el ámbito académico como en el profesional.

- **Alto rendimiento:** Gracias a la definición de los documentos y la creación de índices que hace que las lecturas y escrituras se realicen de forma más rápida.
- **Alta disponibilidad:** MongoDB dispone de servidores replicados con restablecimiento automático maestro.

⁴https://cassandra.apache.org/_/index.html

⁵<http://couchdb.apache.org>

⁶<https://riak.com>

⁷<https://www.mongodb.com/es>

- **Fácil escalabilidad:** Permitiendo distribuir colecciones de documentos entre diferentes máquinas de forma muy sencilla.
- **Indexación:** Similar al concepto de índice en una base de datos relacional, permite crear índices e índices secundarios para mejorar el rendimiento de las consultas.
- **Consultas ad-hoc:** Al igual que en una base de datos relacional, MongoDB da soporte a la búsqueda por campos, consultas de rangos, uso de expresiones regulares... A todo lo anterior, se añade la posibilidad de ejecutar y devolver una función JavaScript definida por el programador.
- **Replicación:** Siguiendo el modelo maestro-esclavo. El maestro puede ejecutar comandos de lectura y escritura mientras que el esclavo solo tiene acceso de lectura y la posibilidad de realizar copias de seguridad. En caso de que el maestro caiga, el esclavo puede elegir un nuevo maestro para mantener el servicio de replicación.
- **Balanceo de carga:** MongoDB es capaz de ejecutarse en múltiples servidores, pudiendo balancear la carga y/o duplicar los datos para mantener el correcto funcionamiento del sistema aunque se produzca un fallo hardware.
- **Almacenamiento de archivos:** Todo lo anterior, facilita que este sistema de base de datos pueda ser usado con un sistema de archivos GridFS con balanceo de carga y replicación.
- **Agregación:** Proporciona operadores de agregación y la posibilidad de utilizar funciones MapReduce para el procesamiento de datos por lotes.
- **Ejecución de Javascript del lado del servidor:** Es posible realizar consultas utilizando JavaScript, de forma que éstas son enviadas directamente a la base de datos para ser ejecutadas.

Todas estas características hacen que, en la actualidad, MongoDB sea uno de los principales sistemas de bases de datos NoSQL elegidos en multitud de aplicaciones como: **almacenamiento y registro de eventos, comercio electrónico, juegos, aplicaciones móviles, almacenes de datos, gestión de estadísticas en tiempo real y cualquier aplicación que requiera llevar a cabo analíticas sobre grandes volúmenes de datos.**

3.3.2. MongoDB: Conceptos básicos, utilidades y herramientas

Aunque MongoDB es un sistema de bases de datos NoSQL con todo lo que ello implica, también ofrece toda la funcionalidad de la que disponen las bases de datos relacionales. Sin embargo, **la estructura de una base de datos MongoDB difiere de la de una base de datos relacional.**

En un servidor MongoDB es posible crear tantas bases de datos como se desee. El concepto de **base de datos** es en este caso equivalente al de **base de datos** en los sistemas relacionales. Una vez creada, la base de datos estará compuesta por una o más **colecciones**. El término colección en el ámbito NoSQL es el equivalente al concepto de **tabla** en los sistemas relacionales. Cada colección, por tanto, estará formada por un conjunto de **documentos** (o incluso ninguno, en cuyo caso la colección estaría vacía). El concepto de documento en NoSQL se corresponde con el concepto de **registro** en una base de datos relacional. Un documento estará compuesto por una serie de **campos**, al igual que lo están los registros de una base de datos relacional. En el ámbito NoSQL, y más concretamente en MongoDB, cada documento viene dado por un archivo **JSON** (<http://www.json.org/>) en el cual, siguiendo una estructura clave-valor se especifican las características de cada documento. El listado 3.11 muestra un ejemplo de documento que define un barco.

Listado 3.11: Definición json de un documento barco

```
1 {  
2   name: 'USS Enterprise-D',  
3   operator: 'Starfleet',  
4   type: 'Explorer',  
5   class: 'Galaxy',  
6   crew: 750,  
7   codes: [10, 11, 12]  
8 }
```

Para la administración del sistema de bases de datos, MongoDB pone a disposición de los usuarios las siguientes utilidades:

- **mongo**: Se trata del shell interactivo de MongoDB que permite insertar, eliminar, actualizar datos y realizar consultas, además de replicar la información, apagar servidores y ejecutar código JavaScript.
- **mongostat**: Herramienta de línea de comandos que muestra las estadísticas de una instancia en ejecución de MongoDB
- **mongotop**: Herramienta de línea de comandos que muestra la cantidad de tiempo empleado en la lectura y escritura de datos por parte de la instancia en ejecución
- **mongosniff**: Herramienta de línea de comandos que permite hacer un rastreo del tráfico de la red que va desde y hacia MongoDB.
- **mongoimport/mongoexport**: Herramienta de línea de comandos para importar y exportar contenido en o desde .json, .csv o .tsv entre otros.
- **mongodump/mongorestore**: Herramienta de línea de comandos para la creación de una exportación binaria del contenido de la base de datos.

Aunque MongoDB tiene una interfaz administrativa accesible desde `http://localhost:28017/` (siempre que se haya iniciado con la opción `mongod --rest`, existen algunas herramientas gráficas para la administración y uso de este sistema de base de datos como **UMongo** (<https://github.com/agirbal/umongo>), el cual

es una aplicación open source de sobremesa para navegar y administrar un clúster MongoDB o **Robomongo** (<https://robomongo.org>), que también es una herramienta gráfica open source que incluye una terminal de comandos completamente compatible con el shell de mongo.

3.3.3. MongoDB: Operaciones CRUD

Cualquier sistema de datos permite definir operaciones básicas como son la creación de tablas e inserción de registros, lectura de datos, actualización y eliminación de registros. Estas operaciones básicas se conocen con el nombre de **CRUD (Create, Read, Update, Delete)**. A continuación, se muestran distintos ejemplos para ilustrar la implementación de estas operaciones en MongoDB.

Una vez iniciada una sesión del Shell de Mongo, es posible crear una base de datos utilizando el siguiente comando.

```
test>use DB_Ejemplo  
switched to db DB_Ejemplo
```

Para crear una colección de documentos “Barco” dentro de la base de datos DB_Ejemplo se puede escribir.

```
DB_Ejemplo>db.createCollection(“ships”)  
{ ok: 1 }
```

Para visualizar las bases de datos almacenadas en el servidor se puede utilizar el comando *show dbs* mientras que para obtener un listado de las colecciones de la base de datos sobre la que se está trabajando, es posible utilizar el comando *show collections*. Por otra parte, para insertar un documento dentro de la colección, como el mostrado en el listado 3.11, se ejecuta el siguiente comando.

```
db.ships.insert({ name:'USS-Enterprise-D',operator:'Starfleet',type:'Explorer',class:  
'Galaxy',crew:750,codes:[10,11,12]})
```

Para actualizar el nombre del barco “USS Prometheus” por “USS Something” es posible escribir el comando.

```
db.ships.update({ name : 'USS Prometheus' }, { name : 'USS Something' })
```

Mientras que si se pretenden establecer o cambiar varios atributos de un mismo documento, como por ejemplo el operador y la clase, se puede utilizar el comando *update* de la siguiente forma.

```
db.ships.update({name : 'USS Something'}, {$set : {operator : 'Starfleet', class : 'Prometheus'}})
```

Finalmente, la eliminación de algún atributo de un documento también es una operación de actualización que puede realizarse de la siguiente forma.

```
db.ships.update({name : 'USS Something'}, {$unset : {operator : 1}})
```

La eliminación de documentos de una colección puede realizarse de forma directa o bien utilizando expresiones regulares. A continuación se muestran dos comandos para ilustrar ambas formas de eliminación. El segundo de ellos elimina aquellos documentos que comienzan por “a”.

```
db.ships.remove({name : 'USS Prometheus'})  
db.ships.remove({name:{$regex:'A*'}})
```

Para leer y mostrar documentos, se utiliza el comando *find()*. A continuación se muestra un ejemplo en el que, el primer comando muestra un documento al azar de los existentes, el segundo muestra todos los documentos y lo hace de forma indexada en lugar de como texto seguido, el tercero muestra solo los nombres de los barcos y el último, encuentra un documento cuyo nombre sea “USS Defiant”.

```
db.ships.findOne()  
db.ships.find().prettyPrint()  
db.ships.find({}, {name:true})  
db.ships.findOne({'name':'USS Defiant'})
```

3.3.4. Mongo DB: Consultas y Agregación

Al igual que cualquier lenguaje de manipulación de datos, MongoDB dispone de las herramientas y operadores necesarios para realizar consultas sobre los documentos almacenados en las colecciones. A continuación, se muestran ejemplos de uso de los principales operadores.

Los operadores relaciones mayor que, menor que, mayor o igual que y menor o igual que se corresponden, respectivamente, con los operadores \$gt, \$lt, \$gte, \$lte. Además, también es posible utilizar el operador \$regex para recuperar elementos que cumplan una expresión regular. A continuación, se muestran dos consultas que recuperan aquellos barcos que permitan subir a bordo a más de cien pasajeros y otra consulta para recuperar aquellos que dejen subir tan solo a 100 pasajeros o menos.

```
db.ships.find({crew:{$gt:100}})

db.ships.find({crew:{$lte:100}})
```

El operador `$exists` permite devolver aquellos documentos para los que existe o no un determinado atributo. El siguiente comando permite encontrar aquellos barcos para los cuales existe el campo “class”.

```
db.ships.find({class:{$exists:true}})
```

Las funciones de agregación son especialmente útiles en los lenguajes de consulta y manipulación de datos. De esta forma, `$sum` permite agregar mediante la operación suma una serie de valores, `$avg` permite obtener la media, `$min` y `$max` encontrar el valor máximo y mínimo de un atributo para el conjunto de elementos, `$push` introduce en un array los resultados de la consulta que se ha realizado, `$addToSet` es similar al anterior solo que sin incluir valores duplicados y, por último, `$first` y `$last` permiten obtener el primer y último documento en una consulta. A continuación, se muestra un ejemplo del uso de cada uno de estos operadores de agregación.

```
db.ships.aggregate([{$group: {_id: "$operator", num_ships: {$sum: "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", num_ships : {$avg : "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", num_ships : {$min : "$crew"}}}])

db.ships.aggregate([{$group : {_id : "$operator", classes : {$push: "$class"}}}])

db.ships.aggregate([{$group : {_id : "$operator", classes : {$addToSet : "$class"}}}])

db.ships.aggregate([{$group: {_id: "$operator", last_class: {$last: "$class"}}}])
```

Finalmente, MongoDB pone a disposición de los usuarios multitud de funciones útiles en la realización de consultas. La tabla 3.4 muestra algunas de las más utilizadas.

Tabla 3.4: Funciones útiles en MongoDB

Función	Significado
<code>\$project</code>	Cambia el conjunto de documentos modificando sus claves y valores
<code>\$match</code>	Operación de filtrado para reducir el número de elementos recuperados
<code>\$group</code>	Operador de agregación para agrupar resultados
<code>\$sort</code>	Ordena los documentos
<code>\$skip</code>	Recupera los documentos a partir de un numero especificado por el usuario
<code>\$limit</code>	Limita los resultados de la consulta al valor pasado como parámetro a la función
<code>\$unwind</code>	Utilizado como equivalente al join de SQL

Bibliografía

- [AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.
- [BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.
- [BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms*. Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.
- [dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.
- [FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.
- [Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.
- [GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.
- [HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

- [IGG03] C. Imhoff, N. Galemme, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.
- [Inm02] William H. Inmon. *Building the Data Warehouse, 3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.
- [JVL03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.
- [KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.
- [MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.
- [MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.
- [PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma, 2006.
- [RB02] Pablo Rodríguez Bocca. *Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos*. Interoperabilidad, Monografía, 2002.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.
- [SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.
- [Tiw11] Shashank Tiwari. *Professional NoSQL*. Wrox Press Ltd., 2011.
- [TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.

Big Data Aplicado

*Conforme a contenidos del «Curso de Especialización
en Inteligencia Artificial y Big Data»*



Universidad de Castilla-La Mancha

Escuela Superior de Informática
Ciudad Real

4

Capítulo

Gestión de Soluciones - IV

Julio Alberto López Gómez

En este capítulo se profundizará en las **bases de datos orientadas a grafos** como solución a problemas de *Big Data*. En la actualidad, cualquier problema puede ser modelizado por medio de grafos. Esta estructura matemática **versátil y escalable** proporciona un mayor rendimiento a los sistemas que utilizan este tipo de bases de datos. Para ilustrar todos los conceptos de este capítulo, se utilizará **Neo4j** como plataforma de bases de datos orientada a grafos.

4.1. Bases de datos orientadas a grafos

Una base de datos orientada a grafos es un tipo de base de datos **NoSQL** como lo son también, por ejemplo, las bases de datos documentales. Un sistema de gestión de bases de datos orientadas a grafos es aquel **define métodos y operaciones CRUD sobre un modelo de datos representado mediante un grafo**. Por norma general, este tipo de bases de datos se implementan para su uso junto con sistemas OLTP, por lo que están diseñadas en términos de relaciones de integridad y optimizadas para ofrecer un rendimiento operacional.

Aunque existen diferentes tipos de modelos de datos basados en grafos, cada sistema de base de datos elegirá uno de ellos para representar el contenido de la base de datos. Cualquiera de estos modelos está inspirado en **la estructura matemática de un grafo para modelizar un conjunto de datos**. Matemáticamente, un grafo es un conjunto de **nodos o vértices**, que representan entidades de un dominio, y un conjunto de **aristas o enlaces**, que representan las relaciones o interacciones que se producen entre las entidades. La figura 4.1 muestra un ejemplo de grafo que representa las relaciones de seguimiento entre distintos canales en YouTube.

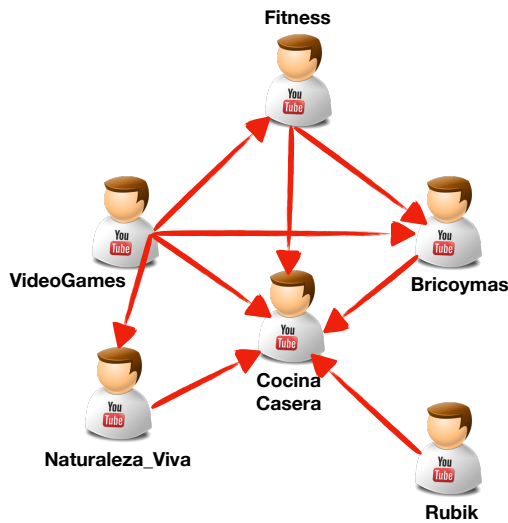


Figura 4.1: Ejemplo de grafo: relaciones de seguimiento entre canales YouTube

Uno de los modelos de datos basado en grafos más popular es **el modelo de grafo de propiedades etiquetadas**. Un grafo representado mediante este modelo, debe cumplir las siguientes características principales: **1)** debe contener un conjunto de nodos y aristas o relaciones entre dichos nodos, **2)** los nodos contienen propiedades, definidas a través de pares “clave-valor”, **3)** los nodos pueden estar etiquetados con una o más etiquetas, **4)** las relaciones entre los nodos están nombradas y son dirigidas, teniendo siempre un nodo de inicio y otro nodo de fin, **5)** las relaciones del grafo también pueden contener propiedades. Este modelo, a pesar de su simplicidad, es sencillo de entender y permite modelar cualquier problema y/o conjunto de datos.

4.1.1. Características principales

El aumento del uso de los sistemas de bases de datos orientados a grafos y el auge que están alcanzando en los últimos años se debe, principalmente, a una serie de características que confieren a estos sistemas una serie de ventajas importantes sobre los sistemas de bases de datos tradicionales y otros sistemas NoSQL.

- **Almacenamiento:** Las bases de datos orientadas a grafo pueden ser de almacenamiento nativo, en cuyo caso los datos se almacenan internamente con estructura de grafo. No obstante, existen bases de datos de grafos que mapean los datos para adaptarlos a una estructura relacional.

- **Procesamiento:** El almacenamiento nativo permite obtener una estructura de datos que no requiere índices de adyacencia para referenciar los nodos del grafo, mejorando el rendimiento en la ejecución de consultas.
- **Rendimiento:** La ejecución de consultas sobre datos estructurados en forma de grafo proporciona escalabilidad al sistema de bases de datos, ya que aunque el número de datos aumente, las consultas se ejecutan sobre la porción del grafo de interés.
- **Flexibilidad:** Los modelos de datos basados en grafos permiten rápidamente añadir nuevos nodos, enlaces y subgrafos a un modelo existente con facilidad y sin pérdida de funcionalidad ni rendimiento.
- **Rapidez:** La naturaleza libre de esquema de las bases de datos orientadas a grafos, juntos con las APIs desarrolladas para su manipulación y los lenguajes de consulta, hacen que estos sistemas sean ágiles y rápidos, integrándolos con las metodologías de desarrollo de software iterativas e incrementales.

4.1.2. Áreas de Aplicación

Las características de las bases de datos orientadas a grafos desarrolladas en el apartado anterior confieren a esta tecnología un sinfín de **aplicaciones reales** en las que su uso mejora significativamente a las tecnologías tradicionales. A continuación, se muestran algunos ejemplos de aplicación donde, actualmente, este tipo de bases de datos es el más ampliamente utilizado.

- **Redes sociales:** El análisis de redes sociales es, a día de hoy, una de las principales fuentes de información de cualquier dominio. El análisis de redes sociales mediante bases de datos orientadas a grafos permite **identificar relaciones explícitas e implícitas** entre usuarios y grupos de usuarios, así como identificar la forma en la que ellos interactúan pudiendo inferir el comportamiento de un usuario en base a sus conexiones. En una red social, dos usuarios presentan una **conexión explícita** si están directamente conectados. Esto ocurre, por ejemplo, entre dos usuarios de Facebook que son amigos o dos compañeros de trabajo de la misma empresa en LinkedIn. Por otra parte, una **relación implícita** es aquella que se produce entre dos usuarios a través de un intermediario, como puede ser otro usuario, un post donde han hecho un comentario, un “like” o un artículo que ambos han comprado.
- **Sistemas de recomendación:** Estos sistemas permiten modelar en forma de grafo las relaciones que se establecen entre personas o usuarios y cosas, como pueden ser productos, servicios, contenido multimedia o cualquier otro concepto relevante en función del dominio de aplicación. **Las relaciones se establecen en función del comportamiento de los usuarios al comprar, consumir contenido, puntuarlo o evaluarlo etc.** De esta forma, los sistemas de recomendación **identifican recursos de interés** para un usuario específico y pueden **predecir su comportamiento al comprar un producto o contratar un servicio**.

- **Información geográfica:** Se trata del caso de aplicación más popular de la teoría de grafos. Las aplicaciones de las bases de datos orientadas a grafos en información geográfica van desde **calcular rutas óptimas entre dos puntos en cualquier tipo de red** (red de carreteras, de ferrocarril, aérea, logística...) **hasta encontrar todos los puntos de interés en un área concreta, encontrar el centro de una región u obtener la intersección entre dos o más regiones, entre otras muchas.** Así, las bases de datos orientadas a grafos permiten modelar estos casos de aplicación como grafos dirigidos sobre los cuales operar para obtener los resultados deseados.

Aunque en este apartado se hayan analizado las principales áreas de aplicación, son muchos los dominios de aplicación donde las bases de datos orientadas a grafos se están convirtiendo en una solución predominante, como lo son **la gestión de datos maestros, gestión de redes y centros de datos o control de acceso entre otros muchos.**

4.2. Neo4j

En esta sección se profundiza en una tecnología concreta de bases de datos orientadas a grafos. Aunque existe un amplio abanico de tecnologías que dan soporte a este tipo de bases de datos, como lo son **OrientDB**¹, **FlockDB**² o **Infinite-Graph**³, en esta sección se profundiza en el trabajo con bases de datos orientadas a grafos utilizando **Neo4j**⁴.

4.2.1. Introducción

Neo4j es una plataforma de bases de datos orientada a grafos lanzada en 2007 e implementada en Java. Entre sus características principales destacan **el almacenamiento y procesamiento de grafos nativo**, lo que la convierte en una alternativa potente para el trabajo en entornos reales, donde se requiere de estas características para mejorar el rendimiento y la escalabilidad de los sistemas. Además, a pesar de lo anterior, Neo4j también **permite trabajar con modelos de datos en forma de grafo de manera transaccional**, por lo que también es útil en sistemas transaccionales.

Junto a todo lo anterior, Neo4j dispone de **amplias librerías que permiten crear también modelos de aprendizaje automático sobre grafos**. Esta plataforma puede integrarse con múltiples lenguajes de programación como Python o C++, entre otros, y ofrece una serie de herramientas muy útiles para usuarios y desarrolladores. A continuación, se enumeran y detallan algunas de ellas:

¹<https://orientdb.org>

²<https://github.com/twitter-archive/flockdb>

³<https://objectivity.com/infinitegraph/>

⁴<https://neo4j.com>

- **Graph Data Science Library:** Se trata de la librería principal que contiene una gran cantidad de algoritmos de búsqueda sobre grafos además de modelos de aprendizaje automático. Se trata de algoritmos y modelos eficientemente programados y escalables para el trabajo con grafos.
- **Neo4j Bloom:** Es una aplicación de visualización y exploración de grafos que permite la visualización de estos desde distintas perspectivas y puntos de vista, ofreciendo así una herramienta visual muy útil para visualizar los resultados de los algoritmos así como mostrar resultados a clientes.
- **Cypher:** Es el lenguaje de creación de consultas utilizado en Neo4j. Se trata de un lenguaje inspirado en SQL, solo que más sencillo y optimizado para el trabajo con grafos.
- **Integradores:** Con el objetivo de integrar y conectar Neo4j con otras plataformas y lenguajes, se han desarrollado distintos conectores para integrar esta plataforma con tecnologías como Apache Spark, Apache Kafka, BI...
- **Herramientas para desarrolladores:** Neo4j dispone de versiones para escritorio y web así como un *sandbox* en el que se proporciona un entorno controlado e integrado de los servicios Neo4j para desarrolladores.
- **Neo4j Aura:** Neo4j ofrece un servicio de computación en la nube para la creación de grafos y ejecución de algoritmos y modelos sobre ellos. Se trata de Neo4j Aura, el cual se encuentra disponible de forma gratuita y está integrado con Google Cloud Platform.

Las características anteriormente mencionadas y la cantidad de herramientas disponibles que Neo4j pone a disposición de los desarrolladores han sido los motivos principales para optar por esta plataforma para ilustrar el trabajo con bases de datos orientadas a grafos.

4.2.2. Importación de datos. Creación de grafos

Para comenzar a trabajar con Neo4j, es necesario **crear un grafo o bien partir de un grafo ya existente** que se importa dentro del área de trabajo de Neo4j. A continuación, se muestra cómo implementar ambos procedimientos.

Importación de un grafo

Sea el caso en el que se pretende importar un grafo a partir de un fichero **.csv**. En el ejemplo que se muestra a continuación, se dispone de dos ficheros **.csv** que contienen, respectivamente, **la definición de los nodos y de las aristas del grafo que se pretende importar**. Así pues, el listado 4.1 muestra los nodos del grafo. Para cada nodo se almacena su identificador, latitud, longitud y población.

Listado 4.1: Definición de nodos de un grafo

```

1 id,latitude,longitude,population
2 "Amsterdam",52.379189,4.899431,821752
3 "Utrecht",52.092876,5.104480,334176
4 "Den Haag",52.078663,4.288788,514861
5 "Immingham",53.61239,-0.22219,9642
6 "Doncaster",53.52285,-1.13116,302400
7 "Hoek van Holland",51.9775,4.13333,9382
8 "Felixstowe",51.96375,1.3511,23689
9 "Ipswich",52.05917,1.15545,133384
10 "Colchester",51.88921,0.90421,104390
11 "London",51.509865,-0.118092,8787892
12 "Rotterdam",51.9225,4.47917,623652
13 "Gouda",52.01667,4.70833,70939

```

Por su parte, el listado 4.2 especifica cada uno de los enlaces entre los nodos del grafo. Para cada uno de los enlaces se identifica el origen y destino del enlace, el tipo de relación que se define entre ellos y el coste.

Listado 4.2: Definición de aristas de un grafo

```

1 src,dst,relationship,cost
2 "Amsterdam","Utrecht","EROAD",46
3 "Amsterdam","Den Haag","EROAD",59
4 "Den Haag","Rotterdam","EROAD",26
5 "Amsterdam","Immingham","EROAD",369
6 "Immingham","Doncaster","EROAD",74
7 "Doncaster","London","EROAD",277
8 "Hoek van Holland","Den Haag","EROAD",27
9 "Felixstowe","Hoek van Holland","EROAD",207
10 "Ipswich","Felixstowe","EROAD",22
11 "Colchester","Ipswich","EROAD",32
12 "London","Colchester","EROAD",106
13 "Gouda","Rotterdam","EROAD",25
14 "Gouda","Utrecht","EROAD",35
15 "Den Haag","Gouda","EROAD",32
16 "Hoek van Holland","Rotterdam","EROAD",33

```

Dados estos dos ficheros, es posible **importar los nodos del grafo** según se indica en el listado 4.3.

Listado 4.3: Importación de nodos

```

1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/transport-nodes.csv"
   AS uri
2 LOAD CSV WITH HEADERS FROM uri AS row
3 MERGE (place:Place {id:row.id})
4 SET place.latitude = toFloat(row.latitude),
5   place.longitude = toFloat(row.longitude),
6   place.population = toInteger(row.population)

```

Así, en el listado anterior se define como identificador de recurso la dirección donde se encuentra el archivo que se pretende cargar, el cual es cargado a través de la instrucción **LOAD CSV**. La instrucción **MERGE** permite definir cada fila como nodo de un grafo de tipo lugar (*place*) y añadir a cada nodo la propiedad *id* cuyo valor será el identificador de la fila en la que se encuentra cada nodo dentro del fichero csv. Finalmente, los valores de latitud y longitud se convierten en valores de tipo real y la población en un valor de tipo entero.

Análogamente a la importación de nodos, el listado 4.4 muestra el código necesario para la **importación del fichero que contiene los enlaces** entre los nodos del grafo.

Listado 4.4: Importación de aristas

```
1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/transport-relationships.csv" AS uri
2 LOAD CSV WITH HEADERS FROM uri AS row
3 MATCH (origin:Place {id: row.src})
4 MATCH (destination:Place {id: row.dst})
5 MERGE (origin)-[:EROAD {distance: toInteger(row.cost)}]->(destination)
```

La diferencia principal de este listado con respecto al anterior, es que en este se necesita identificar mediante la instrucción **MATCH** los nodos de origen y destino de cada uno de los enlaces, lo cual es posible mediante los atributos “src” y “dst” incluidos en el archivo .csv. Finalmente, se utiliza la sentencia **MERGE** para crear una relación entre “origin” y “destination” de tipo **EROAD** que contiene la propiedad “distance” cuyo valor es el coste que aparecía en el fichero .csv.

Creación de un grafo

Otra alternativa es la **creación de un grafo desde cero**, especificando para ello sus nodos y enlaces. La figura 4.2 muestra un ejemplo de un **grafo de co-ocurrencia de hashtags** en el que cada uno de los nodos es un hashtag y los enlaces entre ellos se producen cuando dos hashtags aparecieron en un mismo tweet. Sobre cada enlace, se puede apreciar un valor que indica en cuántos tweets concurrieron los hashtags conectados por medio de dicha arista.



Figura 4.2: Grafo de co-ocurrencia de hashtags

Para representar este grafo en Neo4j, se puede escribir el siguiente fragmento de código que muestra el listado 4.5.

Listado 4.5: Creación grafo de co-ocurrencia de hashtags

```

1 CREATE
2     (JS:Hashtag {name: 'JoaquinSabina'}),
3     (RS:Hashtag {name: 'Rusia2018'}),
4     (AG:Hashtag {name: 'Argentina'}),
5     (FD:Hashtag {name: 'Feliz Domingo'}),
6     (MS:Hashtag {name: 'Messi'}),
7
8     (JS)-[:C00C {ntweet: 52}]->(FD),
9     (FD)-[:C00C {ntweet: 52}]->(JS),
10    (RS)-[:C00C {ntweet: 183}]->(FD),
11    (FD)-[:C00C {ntweet: 183}]->(RS),
12    (RS)-[:C00C {ntweet: 73}]->(AG),
13    (AG)-[:C00C {ntweet: 73}]->(RS),
14    (AG)-[:C00C {ntweet: 112}]->(MS),
15    (MS)-[:C00C {ntweet: 112}]->(AG),
16    (FD)-[:C00C {ntweet: 81}]->(MS),
17    (MS)-[:C00C {ntweet: 81}]->(FD)

```

Tal y como se puede ver, en primer lugar se definen los nodos y sus propiedades y a continuación los enlaces y sus propiedades. **Todo grafo en Neo4j tiene aristas dirigidas, si bien es cierto que cuando se ejecutan las consultas puede no tenerse en cuenta la dirección de las aristas.** Por este motivo, se han especificado las aristas en ambas direcciones. En caso de querer crear una arista que sea interpretada como no dirigida, se debe crear sin el operador flecha que determina el sentido de la arista. Esto es posible hacerlo solo fuera de la instrucción **CREATE**. El listado 4.6 muestra cómo crear la primera relación del grafo solo que de forma no dirigida.

Listado 4.6: Creación de una arista no dirigida

```

1 MATCH(JS:Hashtag{name:'JoaquinSabina'})
2 MATCH(FD:Hashtag{name:'Feliz Domingo'})
3 MERGE (JS)-[:C00C{ntweet:52}]- (FD)

```

Para **mostrar el grafo**, es posible utilizar el siguiente comando. La figura 4.3 muestra cómo quedaría el grafo creado en el listado 4.5.

```
MATCH(n) return (n)
```

Para **eliminar un nodo** creado, es posible utilizar el comando:

```
MATCH(JS:Hashtag{name:'Joaquín Sabina'}) delete (JS)
```

Sin embargo, **un nodo solo puede ser eliminado si se eliminan las relaciones que contiene.** Para ello, es posible utilizar el comando:

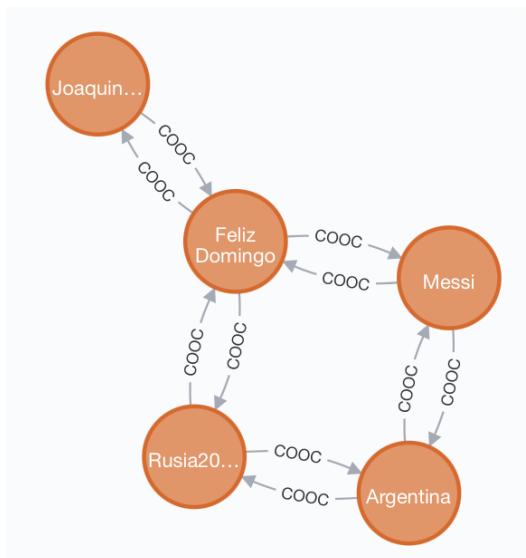


Figura 4.3: Visualización del grafo de co-ocurrencia de hashtags

```
MATCH(JS:Hashtag{name:'Joaquín Sabina'}) detach delete (JS)
```

Mientras que, **para eliminar todos los nodos y aristas**, es posible escribir:

```
MATCH(n) detach delete(n)
```

4.2.3. Recorridos sobre grafos

Los recorridos sobre grafos son una utilidad imprescindible en el trabajo con esta estructura de datos. Del mismo modo, los recorridos también son una herramienta fundamental cuando se trabaja con bases de datos orientadas a grafos.

Un recorrido sobre grafos es un procedimiento sistemático que permite explorar un grafo examinando todos sus vértices y aristas, comenzando desde un vértice inicial. A la hora de recorrer un grafo, existen dos tipos de recorridos: el **recorrido en anchura** (denotado BFS, por sus siglas en inglés) y el **recorrido en profundidad** (denotado DFS, también por sus siglas en inglés).

Recorrido en anchura (BFS)

El recorrido en anchura, también conocido por sus siglas en inglés *Breadth First Search* (BFS) es un procedimiento que permite recorrer un grafo, explorando todos sus nodos y aristas.

El recorrido en anchura parte de un nodo inicial. A continuación, el método comienza a explorar todos los nodos hijo del nodo inicial seleccionado que se encuentran a un salto. Una vez visitados, comienzan a explorarse los nodos hijo del siguiente nivel y así sucesivamente hasta haber recorrido el grafo completo. Este recorrido es muy utilizado cuando se busca un camino con el número mínimo de aristas entre dos vértices dados o cuando se busca un ciclo simple, es decir, una secuencia de nodos y aristas que se pueden describir circularmente en el que todos los nodos y aristas son distintos.

Por ejemplo, sea el grafo de la figura 4.4 que se corresponde con el grafo que se importó anteriormente. Es posible recorrerlo utilizando BFS escribiendo el siguiente código Neo4j que aparece en el listado 4.7.

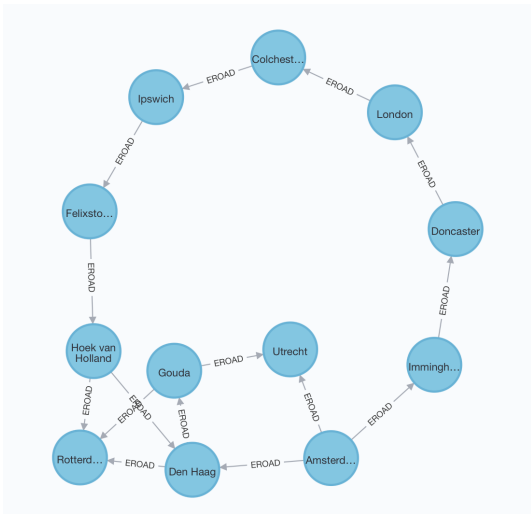


Figura 4.4: Visualización del grafo de ciudades

Listado 4.7: Recorrido en anchura

```
1 CALL gds.graph.create('myGraph', 'Place', 'EROAD', {relationshipProperties: 'distance'})
2 MATCH (a:Place{id: 'Doncaster'})
3 WITH id(a) AS startNode
4 CALL gds.alpha.bfs.stream('myGraph', {startNode: startNode})
5 YIELD path
6 UNWIND [ n in nodes(path) | n.id ] AS tags
7 RETURN tags
```

En el fragmento anterior, se define un grafo llamado *myGraph* el cual está formado por nodos de tipo *Place* y cuyas relaciones son del tipo **EROAD** y tienen una propiedad, denominada *distance*. A continuación, se define el nodo inicial con las sentencias **MATCH** y **WITH**. Después, se llama al método que realiza el recorrido BFS sobre el grafo anterior partiendo desde el nodo inicial para producir

un camino (*path*). Finalmente, se devuelven todos los nodos que forman parte del camino, mostrando los identificadores de cada uno de ellos. El resultado del recorrido es: Doncaster, London, Colchester, Ipswich, Felixstowe, Hoek van Holand, Den Haag, Rotterdam, Gouda, Utrecht.

Recorrido en profundidad (DFS)

El recorrido en profundidad, también conocido por sus siglas en inglés *Depth First Search* (DFS) es un procedimiento alternativo al recorrido en anchura para explorar un grafo, recorriendo todos sus nodos y aristas.

El procedimiento para recorrer un grafo mediante el método DFS es el siguiente: en primer lugar, se parte de un nodo inicial. A partir de él, se empiezan a recorrer todos sus hijos hasta el último nivel. Una vez llegados al último nivel del grafo, se vuelve hacia atrás recorriendo todos los hijos de los nodos de niveles anteriores.

En Neo4j, el procedimiento para recorrer un grafo utilizando el recorrido BFS se muestra en el listado de código 4.8. El procedimiento, como se puede comprobar, es prácticamente idéntico al del listado 4.7 solo que cambiando el método de recorrido. El resultado obtenido es: Doncaster, London, Colchester, Ipswich, Felixstowe, Hoek van Holland, Rotterdam, Den Haag, Gouda, Utrecht.

Listado 4.8: Recorrido en profundidad

```
1 CALL gds.graph.create('myGraph','Place','EROAD',{relationshipProperties:'distance'})
2 MATCH (a:Place{id:'Doncaster'})
3 WITH id(a) AS startNode
4 CALL gds.alpha dfs.stream('myGraph',{startNode: startNode})
5 YIELD path
6 UNWIND [ n in nodes(path) | n.id ] AS tags
7 RETURN tags
```

4.2.4. Caminos mínimos

El cálculo y la obtención de caminos mínimos dentro de un grafo es uno de los problemas clásicos en teoría de grafos con multitud de aplicaciones en el mundo real. **Un camino dentro de un grafo es un conjunto de nodos y aristas que conectan un par de nodos del grafo.** En el caso de los **grafos no pesados**, es decir, aquellos cuyas aristas no tienen coste, el cálculo del camino mínimo se obtiene **minimizando el número de saltos** entre el nodo origen y el nodo destino. En el caso de los **grafos pesados**, aquellos cuyas aristas tienen coste, el cálculo del camino mínimo se obtiene a través del camino en el que **la suma de los costes de sus aristas es mínima**.

Junto con los recorridos de grafos, los métodos de obtención de caminos mínimos son muy útiles cuando se trabaja en **entornos dinámicos**, donde aparecen y desaparecen continuamente aristas del grafo o sus costes cambian dinámicamente. El cálculo de caminos mínimos también es muy útil en aplicaciones que deben dar

respuestas en tiempo real. Algunos ejemplos claros pueden ser encontrar rutas entre dos localidades, como ocurre en Google Maps u obtener los grados de separación entre una empresa y un empleado potencial al que se quiere contratar, como puede ocurrir en LinkedIn.

El **algoritmo de Dijkstra** es uno de los algoritmos más utilizados en teoría de grafos para la obtención de caminos mínimos. Dado el grafo de ciudades con el que se está trabajando a lo largo de este capítulo, el listado 4.9 permite obtener **los caminos mínimos entre un nodo del grafo y todos los demás nodos alcanzables.**

Listado 4.9: Ejecución del algoritmo de Dijkstra sobre el nodo Amsterdam

```

1 MATCH (source:Place{id:'Amsterdam'})
2 CALL gds.allShortestPaths.dijkstra.stream('myGraph', {
3   sourceNode: source,
4   relationshipWeightProperty: 'distance'
5 })
6 YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
7 RETURN
8   index,
9   gds.util.asNode(sourceNode).id AS sourceNodeName,
10  gds.util.asNode(targetNode).id AS targetNodeName,
11  totalCost,
12  [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
13  costs,
14  nodes(path) as path
15 ORDER BY index

```

En el listado anterior, se utiliza la sentencia **MATCH** para definir el nodo de origen y, posteriormente, se guarda el grafo incluyendo el nodo de origen y la propiedad de distancia que es la utilizada para obtener el camino mínimo. A continuación, se producirá un conjunto de caminos en el que se mostrará información completa del nodo de origen, destino, coste total y camino recorrido así como los distintos costes parciales. En caso de querer obtener el camino mínimo entre un par de nodos concretos, es posible utilizar el código mostrado en el listado 4.10 análogo al listado 4.9.

Listado 4.10: Ejecución del algoritmo de Dijkstra sobre el par de nodos Amsterdam-London

```

1 MATCH (source:Place{id:'Amsterdam'}), (target:Place{id:'London'})
2 CALL gds.shortestPath.dijkstra.stream('myGraph', {
3   sourceNode: source,
4   targetNode: target,
5   relationshipWeightProperty: 'distance'
6 })
7 YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
8 RETURN
9   index,
10  gds.util.asNode(sourceNode).name AS sourceNodeName,
11  gds.util.asNode(targetNode).name AS targetNodeName,
12  totalCost,
13  [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
14  costs,

```

```

15     nodes(path) as path
16 ORDER BY index

```

El camino mínimo entre Amsterdam y Londres es el formado por: Amsterdam Immingham, Doncaster, London. Este camino tiene un coste total de 720.

El **método A*** es una **técnica de búsqueda informada** que permite, al igual que el algoritmo de Dijkstra, **calcular el camino mínimo entre dos nodos dados cualesquiera**. Al contrario que el algoritmo de Dijkstra, cuando se va a seleccionar el siguiente nodo del camino, la técnica A* no solo se tiene en cuenta la distancia ya calculada, sino que **este algoritmo combina la distancia calculada con el resultado de una función heurística**. Esta función toma un nodo como entrada y devuelve un valor que corresponde con el coste de llegar al nodo objetivo desde ese nodo. Esta técnica, que se conoce también como de **búsqueda informada**, continúa el recorrido del grafo en cada iteración desde el nodo con el menor coste combinado entre la distancia ya calculada y la función heurística. El listado 4.11 muestra cómo obtener el camino mínimo mediante el método A* entre Amsterdam y Londres.

Listado 4.11: Ejecución del algoritmo A* sobre el par de nodos Amsterdam-London

```

1 CALL gds.graph.create(
2     'myGraph2',
3     'Place',
4     'EROAD',
5     {
6         nodeProperties: ['latitude', 'longitude'],
7         relationshipProperties: 'distance'
8     }
9 )
10 MATCH (source:Place{id:'Amsterdam'}), (target:Place{id:'London'})
11 CALL gds.shortestPath.astar.stream('myGraph2', {
12     sourceNode:source,
13     targetNode:target,
14     latitudeProperty:'latitude',
15     longitudeProperty:'longitude',
16     relationshipWeightProperty: 'distance'
17 })
18 YIELD index, sourceNode, targetNode, totalCost, nodeIds, costs, path
19 RETURN
20     index,
21     gds.util.asNode(sourceNode).id AS sourceNodeName,
22     gds.util.asNode(targetNode).id AS targetNodeName,
23     totalCost,
24     [nodeId IN nodeIds | gds.util.asNode(nodeId).id] AS nodeNames,
25     costs,
26     nodes(path) as path
27 ORDER BY index

```

Como se puede ver, este fragmento de código es similar a los vistos anteriormente, solo que a la hora de guardar el grafo, se almacenan las propiedades de latitud y longitud, necesarias en Neo4j para ejecutar el método A*.

4.2.5. Medidas de centralidad

La centralidad se define como la relevancia de un nodo dentro de un grafo. Por tanto, el estudio de medidas de centralidad permite **identificar nodos relevantes dentro de un grafo**. Esta identificación permitirá entender el comportamiento de la red, qué nodos permiten viralizar con mayor rapidez el contenido de la red, desde qué nodos la información está más accesible, etc. El estudio de medidas de centralidad tiene multitud de aplicaciones, aunque son especialmente notables las relacionadas con los ámbitos de la **publicidad y el marketing**, donde el estudio de estas métricas permite identificar actores relevantes dentro de una red a los que se puede contratar para anunciar un producto o identificar sobre qué actores enviar información para alcanzar a más usuarios.

Respecto a las medidas de centralidad, existe una única medida sino que, en función de los datos y del propósito que se pretende alcanzar, se utilizan unas métricas u otras. a continuación, se van a definir tres medidas de centralidad con las que se realizarán ejemplos en Neo4j: **centralidad de grado, cercanía e intermediación**.

Para trabajar con medidas de centralidad, se va a importar un grafo social que servirá de ejemplo para el cálculo de estas métricas. El listado 4.12 muestra el código necesario para su creación. Por otra parte, la figura 4.5 muestra una visualización del grafo importado.

Listado 4.12: Importación de un grafo social

```
1 WITH "https://github.com/neo4j-graph-analytics/book/raw/master/data/" AS base
2 WITH base + "social-nodes.csv" AS uri
3 LOAD CSV WITH HEADERS FROM uri AS row
4 MERGE (:User {id: row.id})
5
6 WITH base + "social-relationships.csv" AS uri
7 LOAD CSV WITH HEADERS FROM uri AS row
8 MATCH (source:User {id: row.src})
9 MATCH (destination:User {id: row.dst})
10 MERGE (source)-[:FOLLOWS]->(destination)
```

Centralidad de grado

El grado de un nodo en un grafo se define como el número de aristas o conexiones que entran o salen de un nodo. Así pues, en un nodo de un grafo se distinguen dos grados: el **grado de entrada**, correspondiente al conjunto de conexiones que entran a un nodo y el **grado de salida**, que se corresponde con el conjunto de aristas que salen de un nodo. El primero indica la **prominencia** de un nodo dentro de la red, mientras que el segundo la **influencia** del nodo dentro del grafo. Esta medida de centralidad es muy utilizada para identificar actores prominentes o influyentes o para identificar conductas fraudulentas, caracterizadas en ocasiones por una actividad anormal. Para calcular la centralidad de grado en este

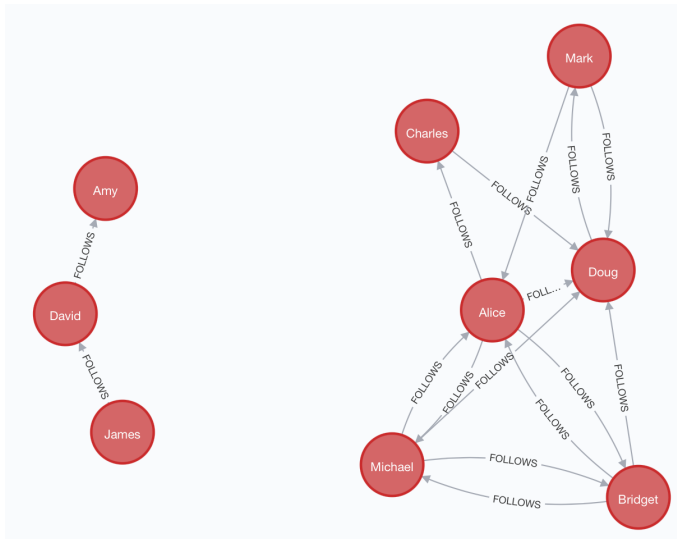


Figura 4.5: Visualización de un grafo social

grafo, se almacenará en primer lugar el grafo creado y, a continuación, se ejecutará el método que calcula la centralidad de grado según el código mostrado en el listado 4.13. El nodo con mayor grado es Doug, que tiene un grado de 5, seguido de Alice que tiene un grado de 3.

Listado 4.13: Cálculo de la centralidad de grado

```

1 CALL gds.graph.create(
2   'myGraph',
3   'User',
4   {
5     FOLLOWS: {
6       orientation: 'REVERSE'
7     }
8   }
9 )
10 CALL gds.degree.stream('myGraph')
11 YIELD nodeId, score
12 RETURN gds.util.asNode(nodeId).id AS name, score AS followers
13 ORDER BY followers DESC, name DESC

```

Cercanía

La cercanía es una medida de centralidad que determina qué nodos del grafo expanden rápida y eficientemente la información a través del grafo. Para calcular esta medida, se obtiene **la suma del inverso de las distancias de un nodo al resto**. De esta forma, dado un nodo u la cercanía del mismo se calcula por medio de la ecuación (4.1).

$$C(u) = \frac{1}{\sum_{v=1}^{n-1} d(u, v)} \quad (4.1)$$

Donde n es el número de nodos del grafo y $d(u, v)$ es la distancia del camino mínimo entre u y v . **La cercanía es una métrica muy utilizada para estimar tiempos de llegada en redes logísticas, para descubrir actores en posiciones privilegiadas en redes sociales o para estudiar la prominencia de palabras en un documento en el campo de la minería de textos.** Para obtener la cercanía en el grafo social de ejemplo, se puede utilizar el fragmento de código mostrado en el listado 4.14. En el grafo de la imagen, Alice, Doug y David tienen una cercanía de 1.

Listado 4.14: Cálculo de la cercanía

```
1 CALL gds.alpha.closeness.stream({
2   nodeProjection: 'User',
3   relationshipProjection: 'FOLLOWS'
4 })
5 YIELD nodeId, centrality
6 RETURN gds.util.asNode(nodeId).name AS user, centrality
7 ORDER BY centrality DESC
```

Intermediación

La intermediación es una medida de centralidad que permite detectar la influencia que tiene un nodo o actor del grafo en el flujo de información o de recursos de la red. El cálculo de la intermediación permite identificar a nodos que hacen de puentes entre distintas porciones del grafo. Esta medida de centralidad **es muy utilizada para la identificación de influencers y el estudio de la viralización de mensajes en redes sociales.** Intuitivamente, la intermediación de un nodo será mayor en tanto en cuanto dicho nodo aparezca en los caminos mínimos de cualquier otro par de nodos. Más formalmente, la intermediación puede calcularse según la ecuación (4.2).

$$B(u) = \sum_{s \neq u \neq t} \frac{p(u)}{p} \quad (4.2)$$

Donde u es el nodo del cual se calcula la intermediación (B), s y t son nodos del grafo, $p(u)$ es el número de caminos mínimos entre s y t que pasan por u y p es el número de caminos mínimos entre s y t . Para el cálculo de la intermediación, se puede utilizar el fragmento de código mostrado en el listado 4.15. Al ejecutar este listado, se obtiene que Alice tiene la mayor intermediación, teniendo un valor de 10.

Listado 4.15: Cálculo de la intermediación

```
1 CALL gds.betweenness.stream('myGraph')
2 YIELD nodeId, score
3 RETURN gds.util.asNode(nodeId).name AS name, score
4 ORDER BY score DESC
```

4.2.6. Detección de comunidades

A la hora de trabajar con grafos reales, que presentan una gran cantidad de nodos y enlaces, muchas veces se pretende **identificar comunidades** dentro del grafo para aplicar algoritmos sobre ellas. **Una comunidad es, por tanto, un conjunto de nodos que presentan más relaciones entre sí que con el resto de nodos fuera de la comunidad.** La detección e identificación de comunidades permite **identificar comportamientos emergentes y de rebaño dentro de una red.** De esta forma, es posible detectar y predecir hábitos de los usuarios. A continuación, se van a aplicar tres métodos de detección de comunidades sobre el grafo social de ejemplo que se ha utilizado en secciones anteriores.

Conteo de triángulos

Un triángulo es un conjunto de tres nodos que tienen relaciones entre sí. El conteo de triángulos para un nodo dado, permite estudiar o inspeccionar de forma global un grafo y, aplicado sobre componentes conexas, permite inspeccionar regiones de un grafo. Para aplicar este método, es necesario almacenar el grafo teniendo en cuenta que las aristas deben almacenarse como **UNDIRECTED**. El listado 4.16 muestra el fragmento de código necesario para aplicar este método sobre el grafo social de ejemplo. La ejecución de este método da como resultado que Alice y Doug son aquellos que pertenecen a más triángulos, un total de 5.

Listado 4.16: Ejecución del conteo de triángulos

```
1 CALL gds.graph.create(
2   'myGraph2',
3   'User',
4   {
5     FOLLOWS: {
6       orientation: 'UNDIRECTED'
7     }
8   }
```



```

9 )
10 CALL gds.triangleCount.stream('myGraph')
11 YIELD nodeId, triangleCount
12 RETURN gds.util.asNode(nodeId).name
13 AS name, triangleCount
14 ORDER BY triangleCount DESC

```

Coeficiente local de clustering

Este coeficiente **proporciona una medida cuantitativa del grado de agrupación de un nodo**. Para el cálculo del coeficiente local de clustering se utiliza el conteo de triángulos, de forma que se compara el grado de agrupación de un nodo con el grado máximo de agrupación que podría tener. Así pues, el coeficiente local de clustering se puede calcular mediante la ecuación (4.3).

$$CC(u) = \frac{2R_u}{k_u(k_u - 1)} \quad (4.3)$$

Donde u es un nodo, $R(u)$ es el número de relaciones que a través de los vecinos de u (lo cual puede medirse con el número de triángulos que pasan por u) y $k(u)$ es el grado de u . El cálculo del coeficiente local de clustering puede realizarse por medio del listado 4.17. Los actores Bridget, Charles, Mark y Michael tienen el mayor coeficiente local de clustering, que es 1.

Listado 4.17: Cálculo del coeficiente local de clustering

```

1 CALL gds.localClusteringCoefficient.stream('myGraph2')
2 YIELD nodeId, localClusteringCoefficient
3 RETURN gds.util.asNode(nodeId).id
4 AS name, localClusteringCoefficient
5 ORDER BY localClusteringCoefficient DESC

```

Por último, **el coeficiente global de clustering se calcula como la suma normalizada de los coeficientes de clustering locales**. Así, estos coeficientes nos permiten encontrar medidas cuantitativas para detectar comunidades, pudiendo especificar incluso un umbral para establecer la comunidad (por ejemplo, especificar que los nodos han de estar conectados en un 40 %).

Componentes fuertemente conexas

En un grafo dirigido, una componente fuertemente conexa es aquel grupo de nodos en el que cualquier nodo puede ser alcanzado por cualquier otro en ambas direcciones. El estudio de las componentes fuertemente conexas en un grafo permite **estudiar la conectividad de la red**. Para realizar el cálculo de las

componentes conexas, el cual se realiza en un tiempo de procesamiento proporcional al número de nodos, es posible utilizar el listado 4.18. El resultado permite comprobar que hay un total de cuatro componentes fuertemente conexas en el grafo social.

Listado 4.18: Obtención de componentes fuertemente conexas

```
1 CALL gds.alpha.scc.stream({
2   nodeProjection: 'User',
3   relationshipProjection: 'FOLLOWS'
4 })
5 YIELD nodeId, componentId
6 RETURN gds.util.asNode(nodeId).id AS Name, componentId AS Component
7 ORDER BY Component DESC
```

4.2.7. Predicción de enlaces

Los grafos son estructuras de datos que representan **sistemas dinámicos**, que evolucionan a lo largo del tiempo. Por este motivo, es muy común que en un grafo cualquiera aparezcan y desaparezcan nuevos nodos y conexiones entre dichos nodos. **Los métodos de predicción de enlaces permiten predecir qué enlaces se formarán próximamente entre los nodos del grafo, permitiendo adelantarse a los acontecimientos y prevenir eventualidades.** Como norma general, los métodos de predicción de enlaces **se basan en medidas de cercanía y de centralidad** asumiendo que los nuevos enlaces se producirán, mayoritariamente, sobre/desde los nodos más relevantes. A continuación, se detalla el funcionamiento de tres métodos comúnmente utilizados en predicción de enlaces.

Vecinos comunes

El método de vecinos comunes se basa en la idea genérica de que dos actores de la red que tienen una relación con un usuario común tendrán más posibilidad de conectarse entre sí que quienes no. Formalmente, dados dos nodos, la posibilidad de que se produzca un enlace entre los nodos x e y viene dada por la ecuación 4.4.

$$CN(x, y) = |N(x) \cap N(y)| \tag{4.4}$$

Donde $N(x)$ es el conjunto de nodos adyacentes a x y $N(y)$ es el conjunto de nodos adyacentes a y . Cuanto mayor es el valor de CN calculado, existe mayor posibilidad de que se produzca un nuevo enlace entre x e y . En el grafo social de ejemplo, el cálculo de vecinos comunes para Charles y Bridget se puede realizar a través del listado 4.19. El resultado de este cálculo es 2.

Listado 4.19: Obtención del valor de vecinos comunes para Charles y Bridget

```
1 MATCH (x:User{id:'Charles'})
```

```

2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.commonNeighbors(x,y) AS score

```

Adhesión preferencial

Este método se basa en la idea general de que cuanto más conectado está un nodo, es más probable que reciba nuevos enlaces. Formalmente, esta idea se puede expresar por medio de la ecuación 4.5

$$CN(x, y) = |N(x) \cdot N(y)| \quad (4.5)$$

En el grafo social de ejemplo, el cálculo de adhesión preferencial para Charles y Bridget se puede realizar a través del listado 4.20. El resultado de este cálculo es 10.

Listado 4.20: Obtención del valor de adhesión preferencial para Charles y Bridget

```

1 MATCH (x:User{id:'Charles'})
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.preferentialAttachment(x, y) AS score

```

Asignación de recursos

Se trata de una métrica compleja que evalúa la cercanía de un par de nodos para determinar la posibilidad de que, entre ellos, se produzca un nuevo enlace. Para ello, se utiliza la expresión dada en la ecuación 4.6.

$$RA(x, y) = \sum_{U \in N(x) \cap N(y)} \frac{1}{N(u)} \quad (4.6)$$

En el grafo social de ejemplo, el cálculo de la asignación de recursos para Charles y Bridget se puede realizar a través del listado 4.21. El resultado de este cálculo es 0.309.

Listado 4.21: Obtención del valor de asignación de recursos para Charles y Bridget

```

1 MATCH (x:User{id:'Charles'})
2 MATCH (y:User{id:'Bridget'})
3 RETURN gds.alpha.linkprediction.resourceAllocation(x, y) AS score

```

Bibliografía

- [AN95] Agnar Aamodt and Mads Nygård. Different roles and mutual dependencies of data, information, and knowledge — an ai perspective on their integration. *Data & Knowledge Engineering*, 16(3):191–222, 1995.
- [BELP13] Ulrik Brandes, Markus Eiglsperger, Jürgen Lerner, and Christian Pich. Graph markup language (graphml). In Roberto Tamassia, editor, *Handbook on Graph Drawing and Visualization*, pages 517–541. Chapman and Hall/CRC, 2013.
- [BNCD16] Rajkumar Buyya, Rodrigo N. Calheiros, and Amir Vahid Dastjerdi. *Big Data : Principles and Paradigms*. Elsevier Science & Technology, 50 Hampshire Street, 5th Floor, Cambridge. USA, 2016.
- [dMP99] Adoración de Miguel and Mario Piattini. *Fundamentos y Modelos de Bases de Datos*. Alfaomega, 1999.
- [FPSS96] Usama Fayyad, Gregory Piatetsky-Shapiro, and Padhraic Smyth. From data mining to knowledge discovery in databases. *AI magazine*, 17(3):37, 1996.
- [Fri19] J. Friesen. *Java XML and JSON: Document Processing for Java SE*. Apress, 2019.
- [GR09] Matteo Golfarelli and Stefano Rizzi. *Data Warehouse Design: Modern Principles and Methodologies*. Mcgraw-hill, 2009.
- [HT14] Francisco Herrera Triguero. *Inteligencia Artificial, Inteligencia Computacional y Big Data*. Servicio de publicaciones. Universidad de Jaén, Jaén, 2014.

- [IGG03] C. Imhoff, N. Galemme, and J.G. Geiger. *Mastering Data Warehouse Design: Relational and Dimensional Techniques*. Wiley, 2003.
- [Inm02] William H. Inmon. *Building the Data Warehouse, 3rd Edition*. John Wiley & Sons, Inc., USA, 3rd edition, 2002.
- [JVV03] Matthias Jarke, Yannis Vassiliou, Panos Vassiliadis, and Maurizio Lenzerini. *Fundamentals of Data Warehouses*. Springer-Verlag, Berlin, Heidelberg, 1st edition, 2003.
- [KR02] Ralph Kimball and Margy Ross. *The Data Warehouse Toolkit: The Complete Guide to Dimensional Modeling*. Wiley, 2 edition, April 2002.
- [MSC13] Cukier Mayer Schönberger, Viktor and Kenneth Cukier. *Big data. La revolución de los datos masivos*. Turner Publicaciones, Madrid, 2013.
- [MTK06] J. Mundy, W. Thornthwaite, and R. Kimball. *The Microsoft Data Warehouse Toolkit: With SQL Server 2005 and the Microsoft Business Intelligence Toolset*. Wiley, 2006.
- [PVMCV06] Mario Piattini Velthuis, Esperanza Marcos, Coral Calero, and Belén Vela. *Tecnología y diseño de bases de datos*. Ra-Ma, 2006.
- [RB02] Pablo Rodríguez Bocca. *Lenguajes canónicos para la descripción de grafos: Estudio y transformación entre esquemas de distintos modelos de datos*. Interoperabilidad, Monografía, 2002.
- [RWE15] Ian Robinson, Jim Webber, and Emil Eifrem. *Graph Databases*. O'Reilly, 2 edition, 2015.
- [SKS06] Abraham Silberschatz, Henry F. Korth, and Sudarshan S. *Fundamentos de bases de datos*. McGraw-Hill, Edificio Valrealty. 1 Planta. Basauri 17. 28023 Aravaca (Madrid), 2006.
- [Tiw11] Shashank Tiwari. *Professional NoSQL*. Wrox Press Ltd., 2011.
- [TR03] Erik T. Ray. *Learning XML*. O'Reilly, 2003.